

Abstraction and refinement for modelling and analysis in cyber physical system design

Michael Butler

TASE 2026, Shanghai, 6 July 2026

Overview

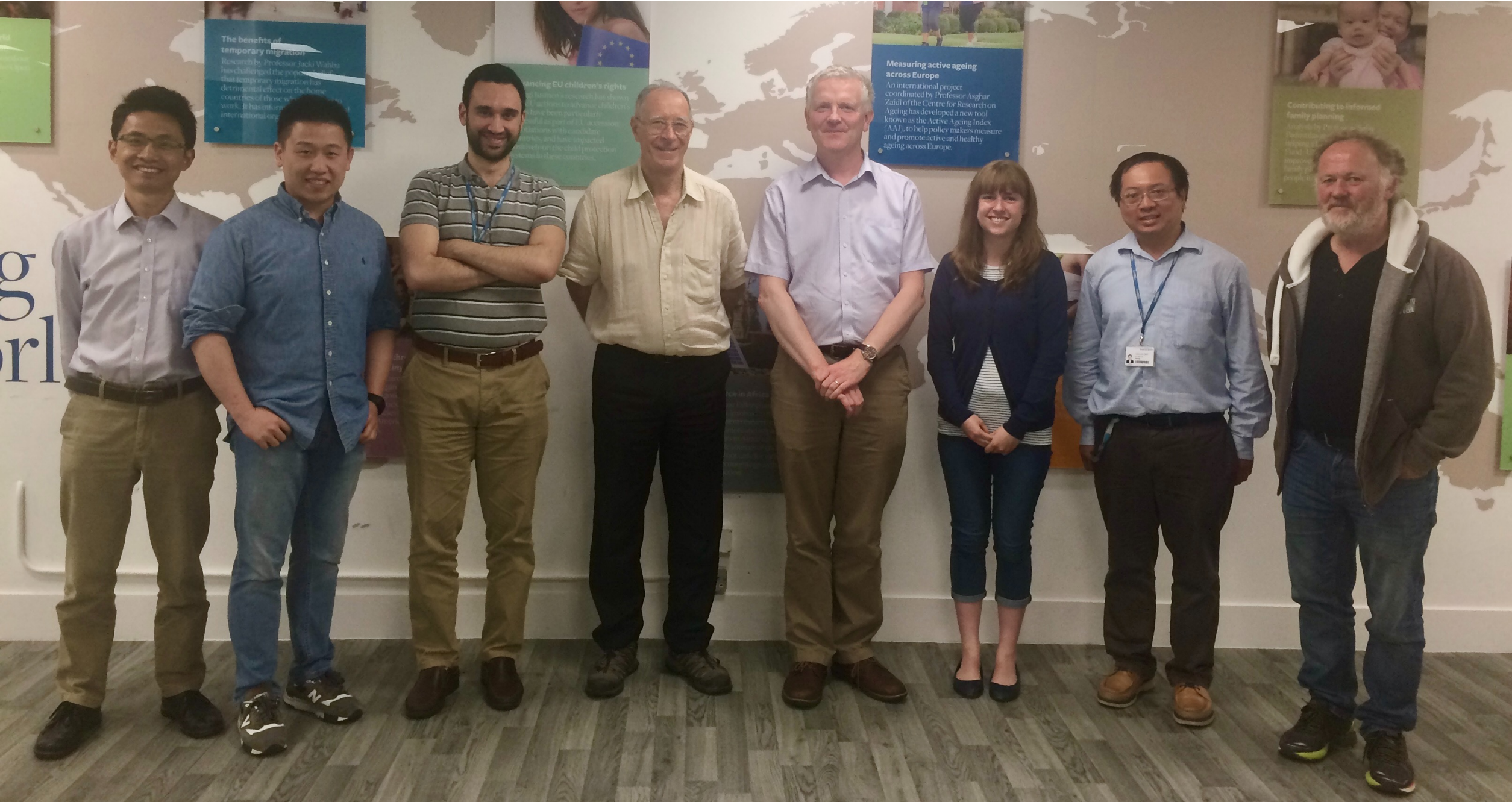
- AI challenges established notions of correctness
- Guaranteed Safe AI: safety specification, world model, verifier.
 - Constructing specifications and world models for complex systems is challenging

This talk

- Guidance on abstraction and refinement to address system complexity
- Focus on cyber-physical system (CPS) safety rather than broader uses of AI
- Role of invariants and proof
- Connection to conventional safety analysis
- Autonomy in robotic systems leads to weaker control
- Approach is established for discrete abstractions of CPS
 - Tentative ideas on continuous abstractions

Acknowledgements

- Asieh Salehi Fathabadi, Colin Snook, Dana Dghaym, Haider Al-Shareefy, Reza Rezazadeh, and Thai Son Hoang
- Fathabadi, Snook, Dghaym, Hoang, Alotaibi, Butler. *Systematic hierarchical analysis of requirements for critical systems*, Innovations Syst Softw Eng 2024
- Fathabadi, Hoang, Butler, *SHARCS: refinement-centric hazard analysis of requirements for critical systems*, ABZ 2026
- HD-Sec Project (UKRI Digital Security by Design programme)



Guaranteed Safe AI system

1. The ***probabilistic safety specification*** encodes societal risk criteria, which should ideally be determined by collective deliberation.
2. The ***verifier*** provides a quantitative guarantee ... that the AI system satisfies the specification with respect to the ***world model***.
3. All potential future effects of the AI system and its environment **relevant to the safety specification** should be modelled (and – if feasible – conservatively **overapproximated**) by the world model.

- *Towards Guaranteed Safe AI: A Framework for Ensuring Robust and Reliable AI Systems*
Dalrymple, Skalse, Bengio, Russell, Tegmark, Seshia, Omohundro, Szegedy, Goldhaber, Ammann, Abate, Halpern, Barrett, Zhao, Tan, Wing, Tenenbaum



Modeling in Event-B

System and
Software
Engineering

Jean-Raymond Abrial

CAMBRIDGE

Prologue: Faultless systems – yes we can!

Modeling vs. programming

Programming is the activity of constructing a piece of formal text that is supposed to instruct a computer how to fulfil certain tasks. *Our intention is not to do that.* What we intend to build is a system within which there is a certain piece of software (the one we shall construct), which is a component among many others. This is the reason why our task is not limited to the software part only.

In doing this as engineers, we are not supposed to instruct a computer; rather, we are supposed to instruct ourselves. To do this in a rigorous way, we have no choice but to build a complete *model* of our future system, including the software that will eventually be constructed, as well as its environment, which, again, is made of equipment, varying physical phenomena, other software, and even users. Programming languages are of no help in doing this. All this has to be carefully modeled so that the exact assumptions within which our software is going to behave are known.

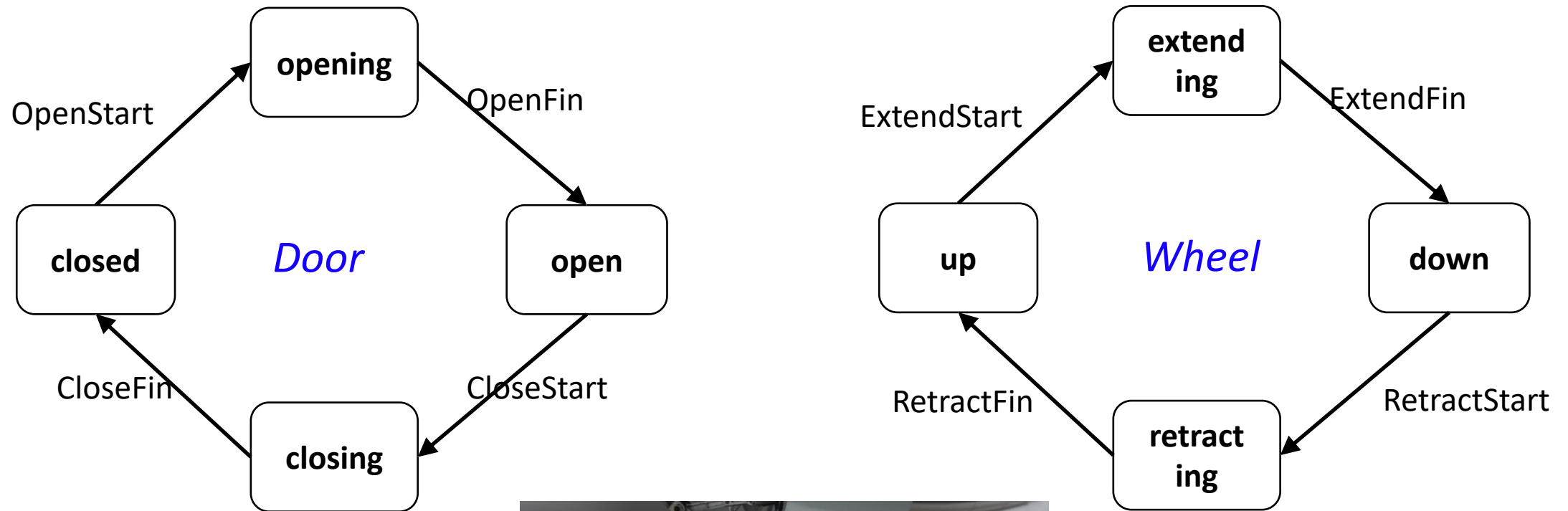
Modeling is the main task of system engineers. Programming then becomes a sub-task which might very well be performed automatically.

Computing has been done in the past (and still is) with the help

System-level invariants

- System consists of multiple components
 - physical and cyber
- System state is a combination of the states of each component
 - product of states
- Many combinations of states are *undesirable*
 - E.g., *elevator* is moving and *door* is open
- System-level invariants:
 - Relationships between states of components
 - Specify desirable states, exclude undesirable states

Abstract model of landing gear



System-level invariant for landing gear

invariant $\text{wheelstate} \neq \text{up} \Rightarrow \text{doorstate} = \text{open}$

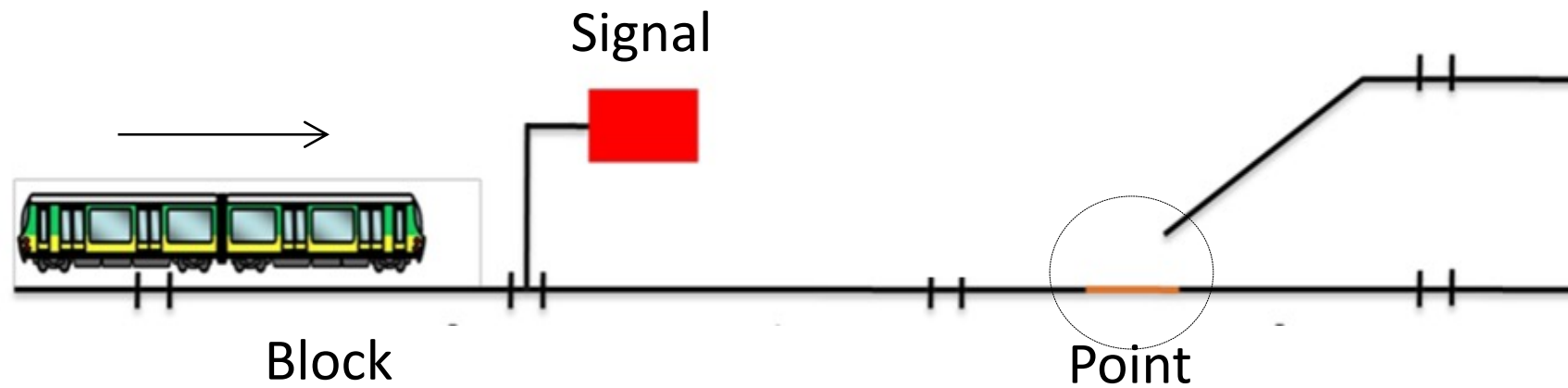
ExtendStart $\triangleq$

when $\text{wheelstate} = \text{up} \wedge \text{doorstate} = \text{open}$

then $\text{wheelstate} := \text{extending}$

Guard needed to
avoid unsafe state

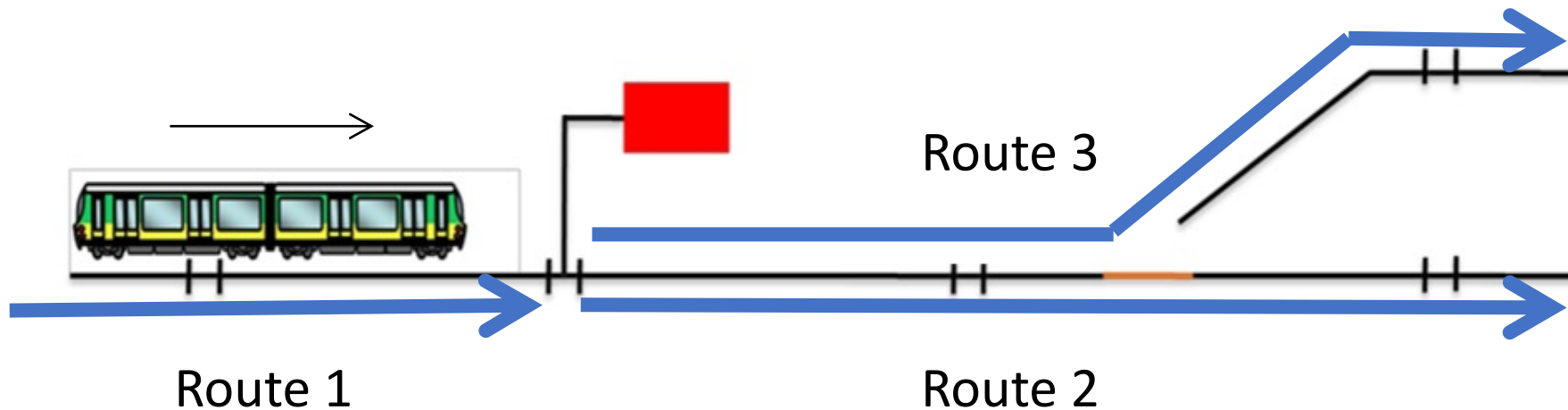
Railway interlocking: Physical components



Domain concept: Route

Route is a *Domain phenomena*:

Used to define problem but no physical manifestation



System-level safety invariant

IF

a signal s is green

THEN

there is some route r that is locked,
for which s is the entry point, and
no block of r is occupied

$\forall s \bullet s \in \text{signal} \wedge \text{colour}(s) = \text{GREEN}$

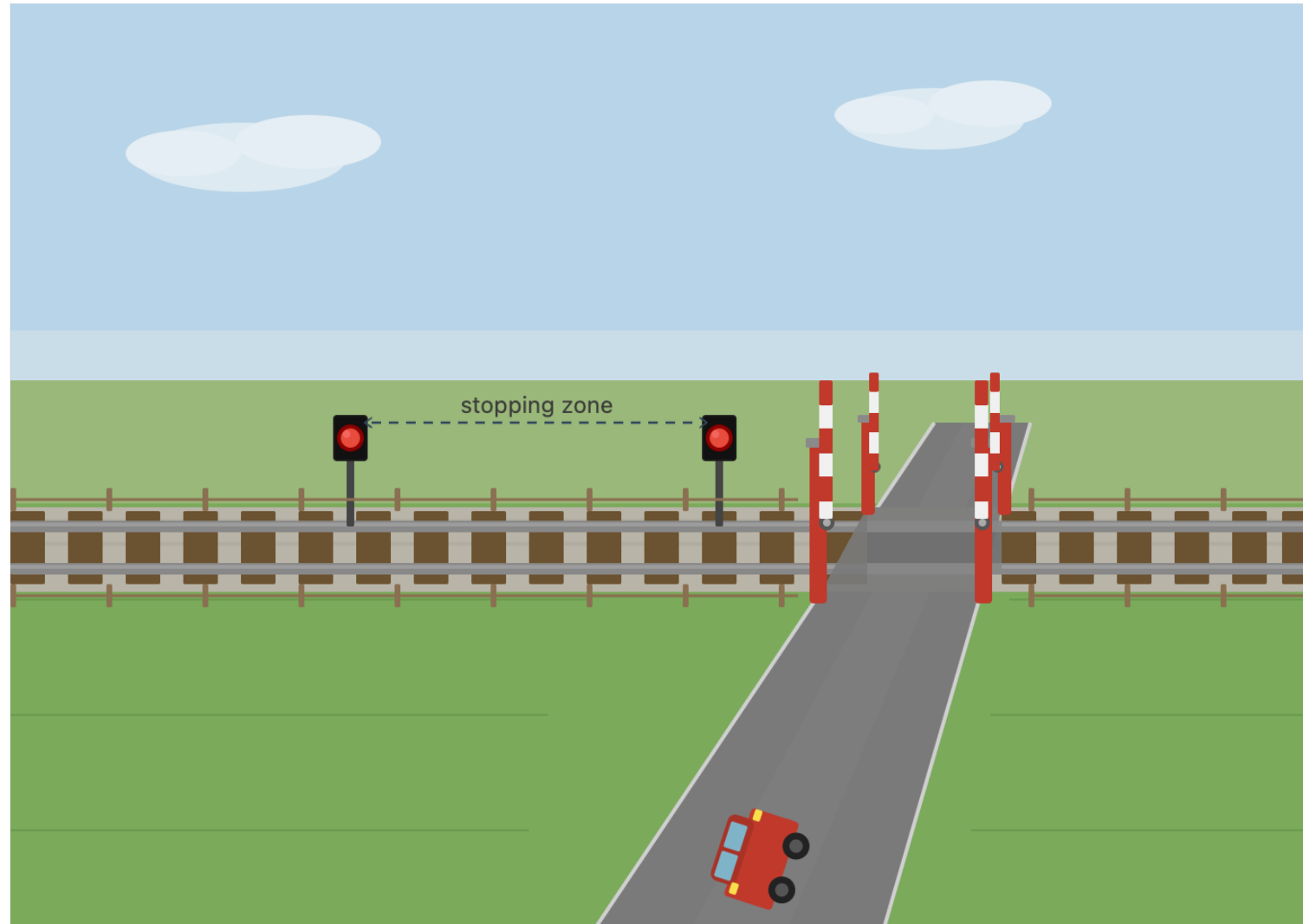
$\Rightarrow$

$\exists r \bullet r \in \text{locked} \wedge \text{entry_signal}(r) = s \wedge$
 $\text{blocks}(r) \cap \text{occupied} = \emptyset$

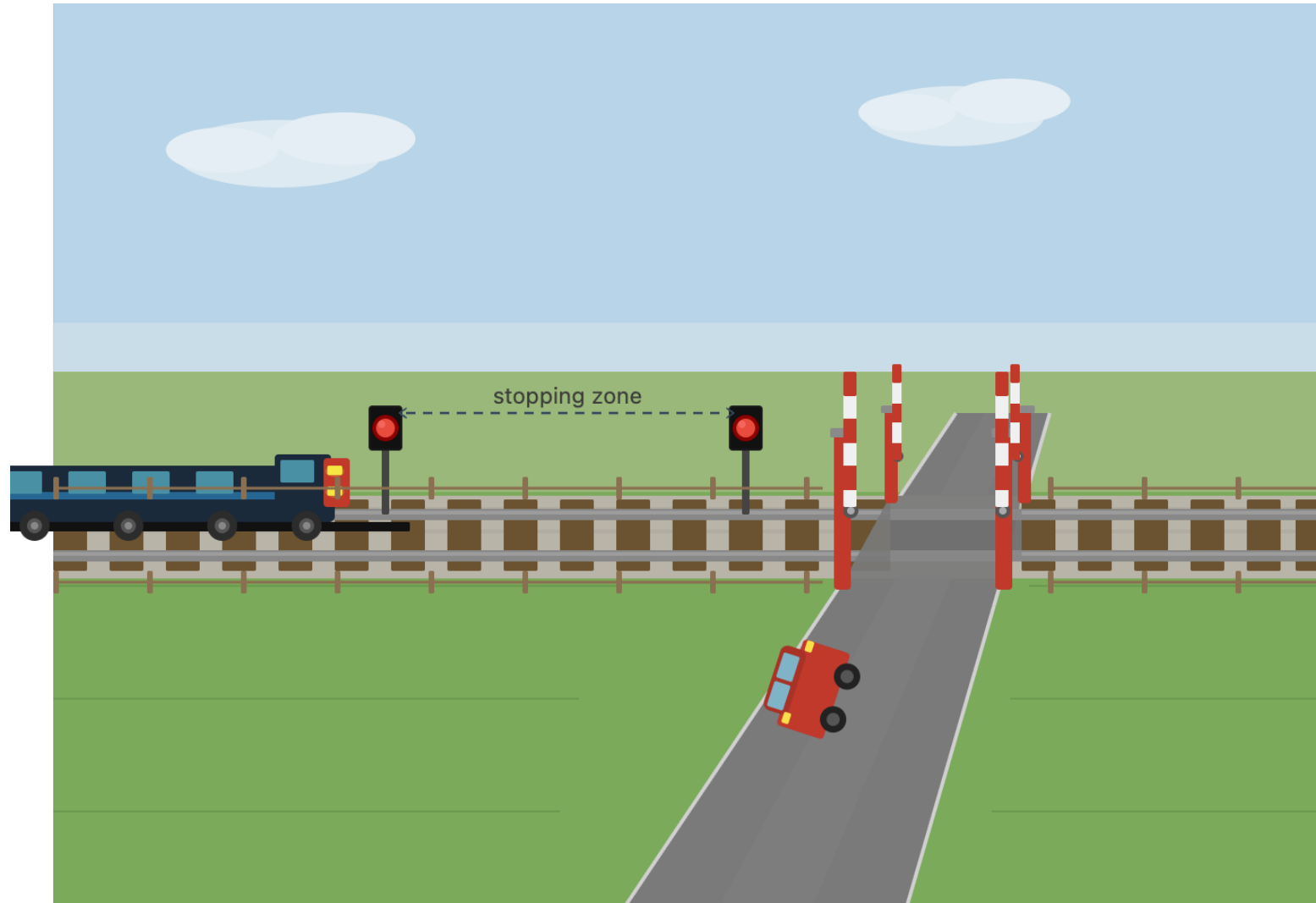
Basic Principles: Refinement

- Refinement is fundamental in engineering practice
- Because a model cannot be defined in a single step
- Models are thus constructed gradually
- From an initial very abstract view to a final concrete one
- Abstraction is usually very difficult to master by informaticians
- So, practitioners have to be seriously educated on this matter

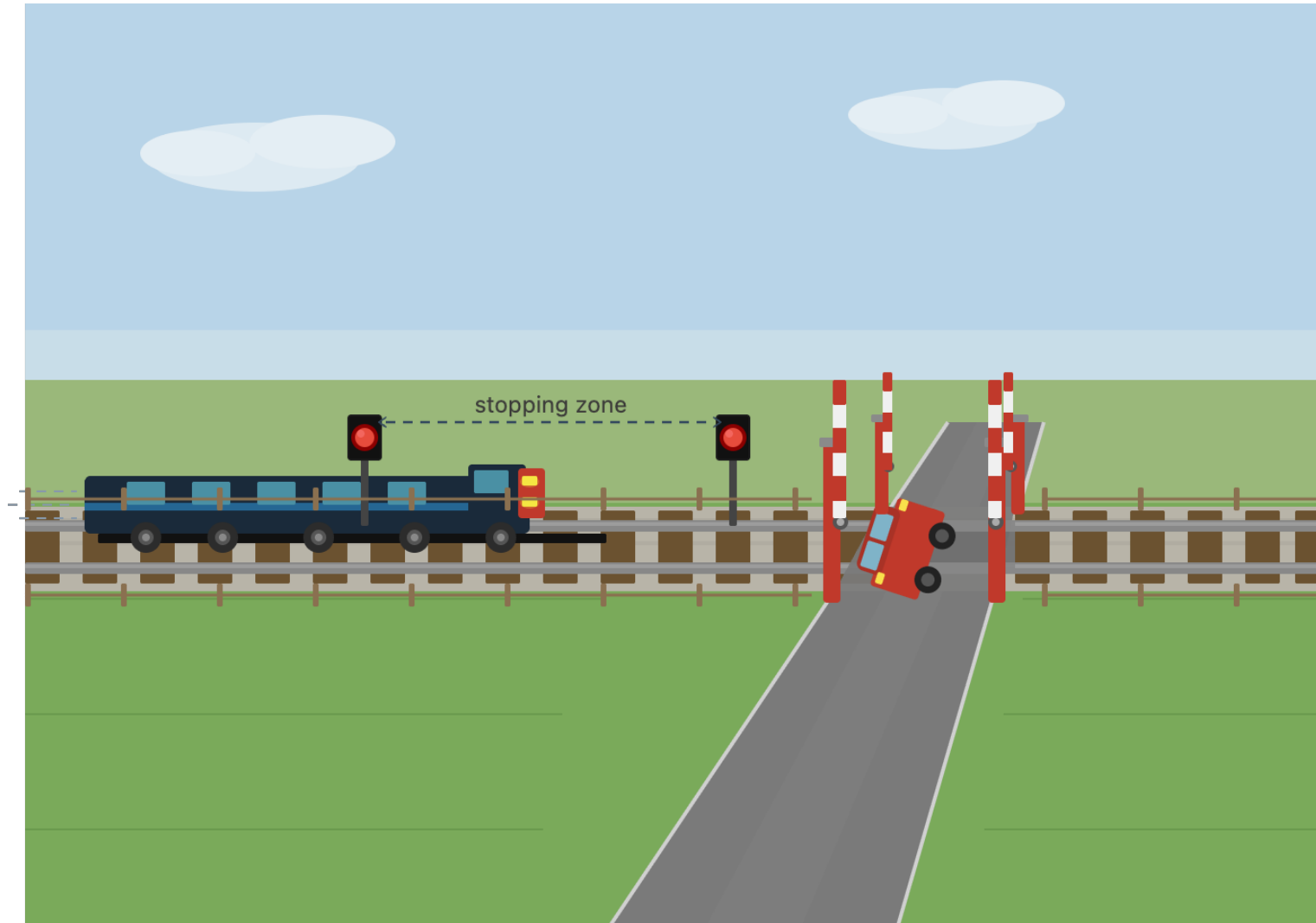




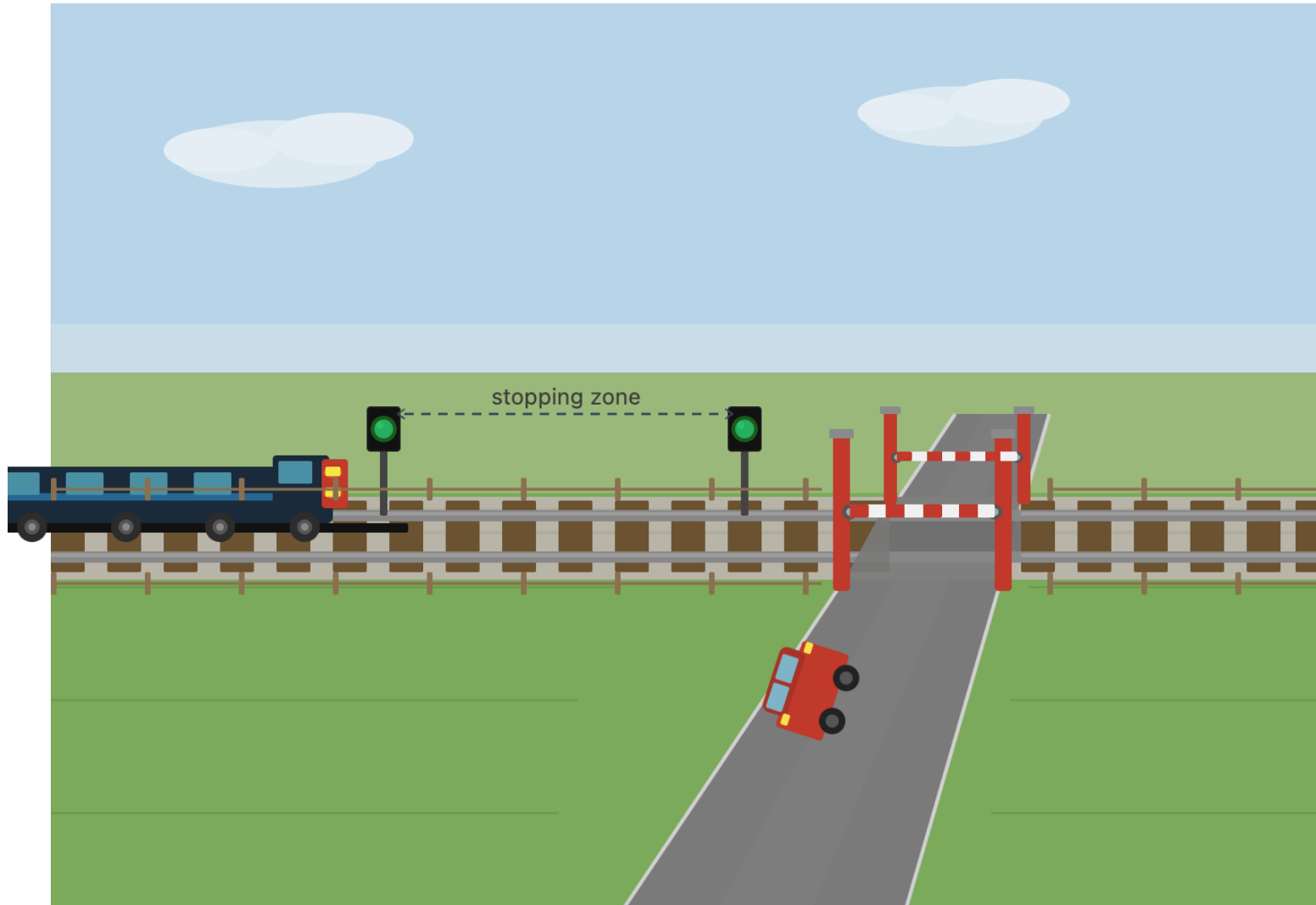
*Graphics created using Claude AI



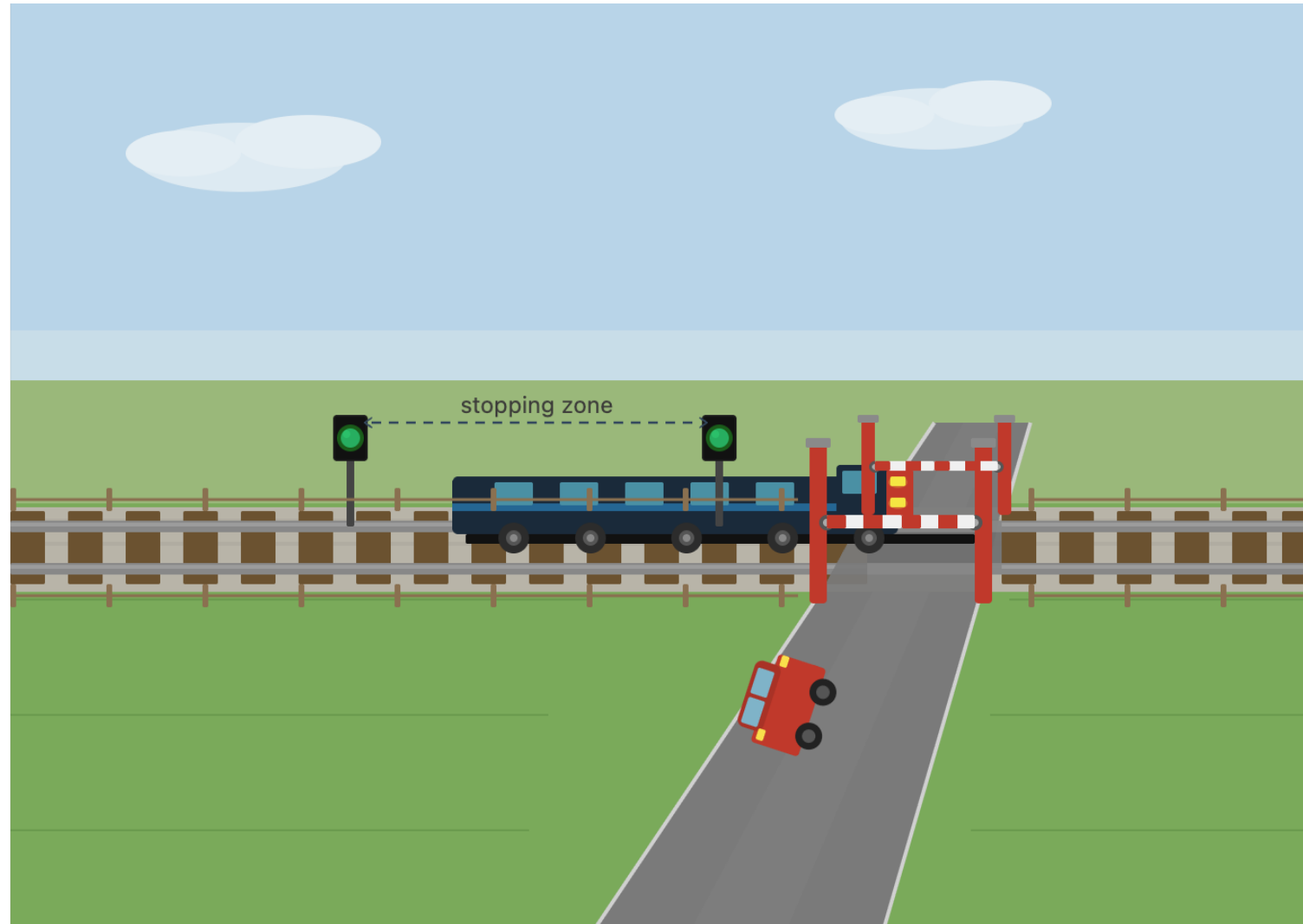
Safe for car to proceed



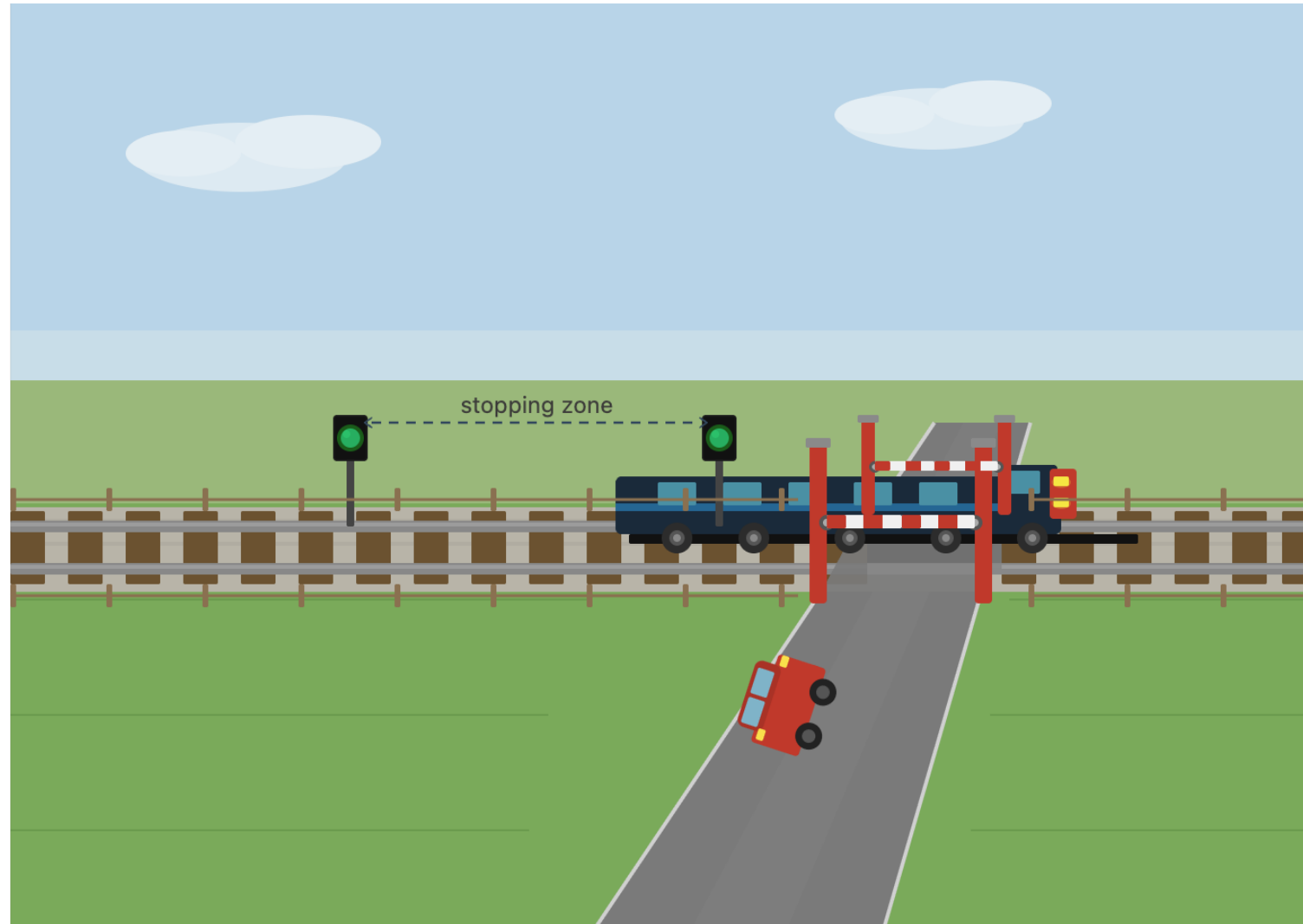
Safe for car to proceed



Safe for train to proceed

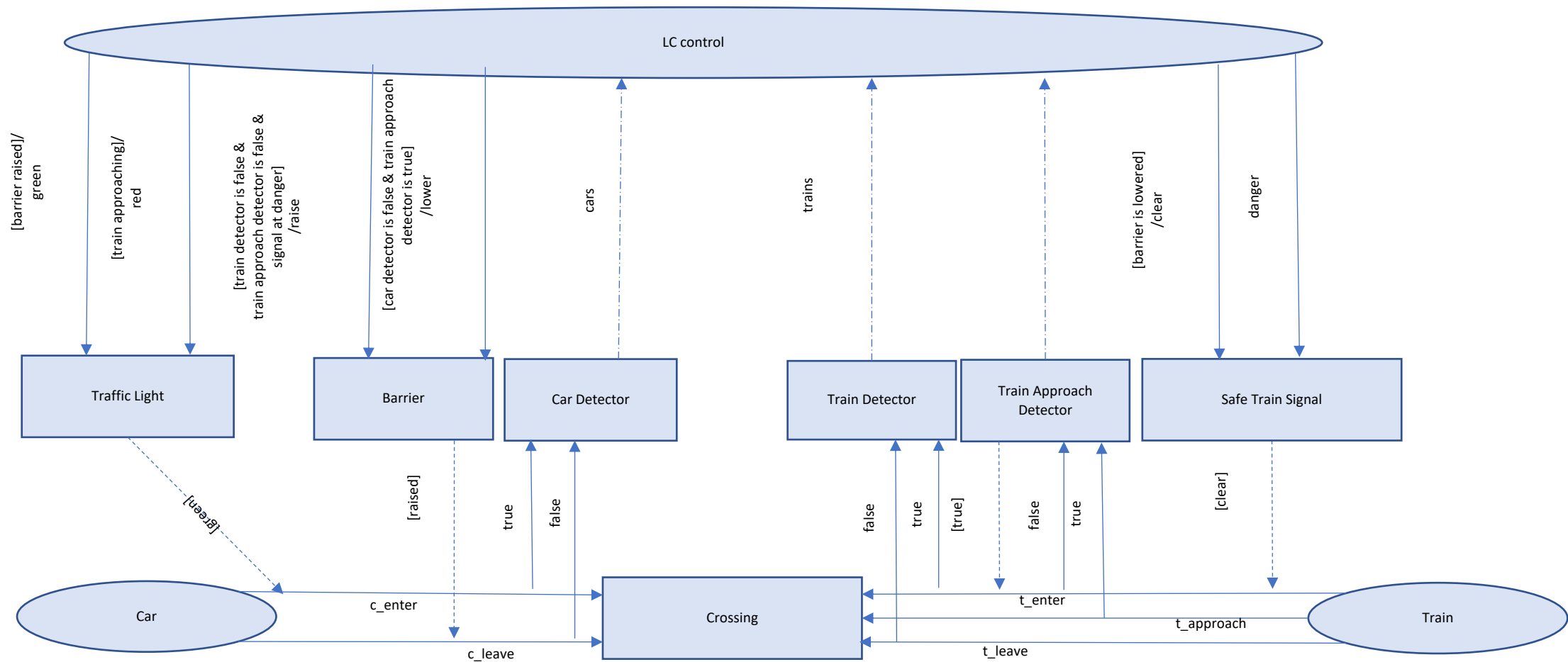


Safe for train to proceed

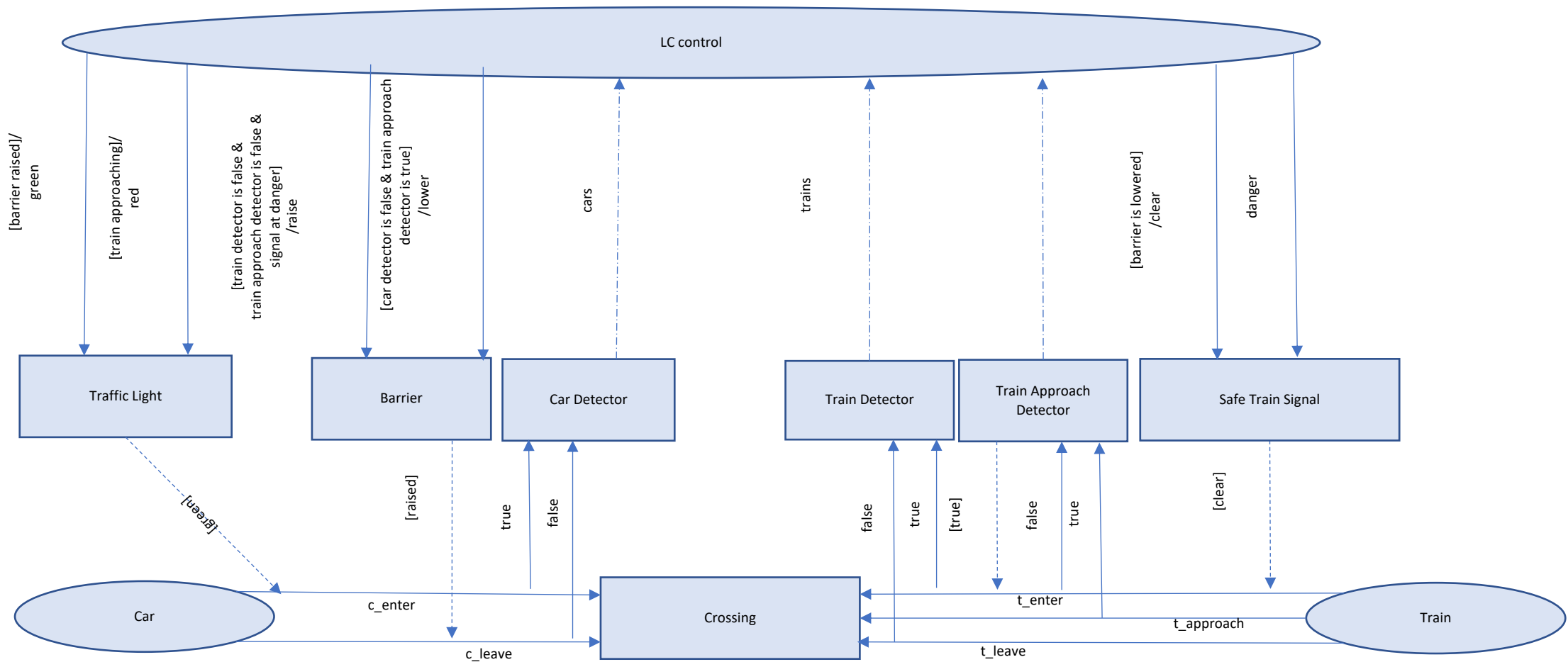


Safe for train to proceed

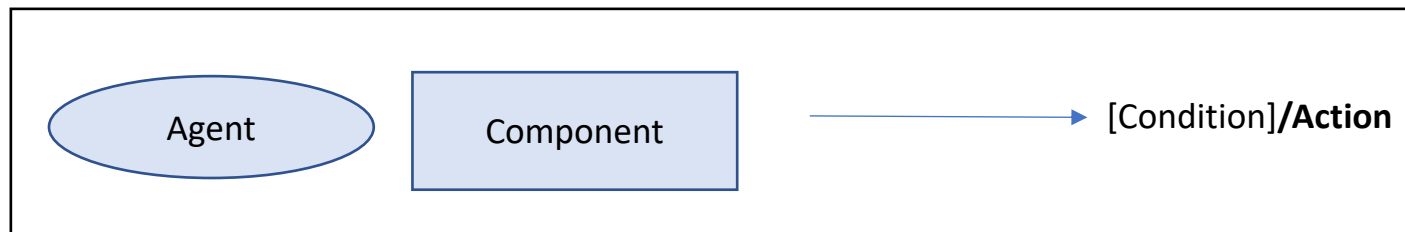
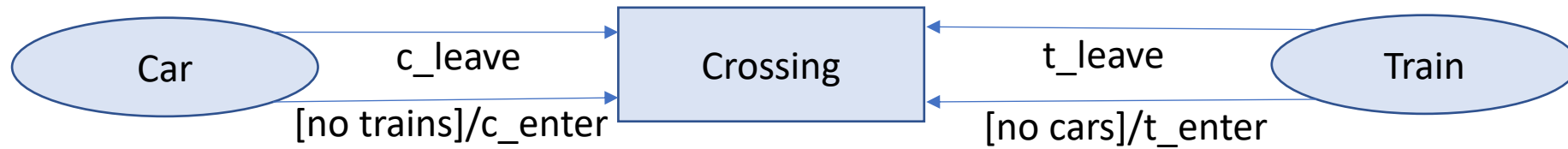
Control structure diagram of Level Crossing



Safety purpose: Prevent collision between **car** and **train** on **crossing** - which elements are needed to model this purpose?



Control abstraction diagram



Preventing Failure at Level Crossing

- **Safety purpose:** Prevent collision between cars and trains
- **Over-approximation:** Prevent car and train on crossing simultaneously
- Failure is negation of safety: A train and a car on the crossing
- Failure prevention:
 - Abstract: add guards to ensure invariants hold (thus preventing failure)
 - Refinement: add mechanisms to enforce abstract guards (thus preventing failure), e.g., signal, barrier

Abstract model

- invariant
not(car on crossing $\wedge$ train on crossing)

CarEnterLC $\triangleq$

when *car* = off_crossing $\wedge$
train = off_crossing
then *car* := on_crossing

TrainEnterLC $\triangleq$

when *train* = off_crossing $\wedge$
car = off_crossing
then *train* := on_crossing

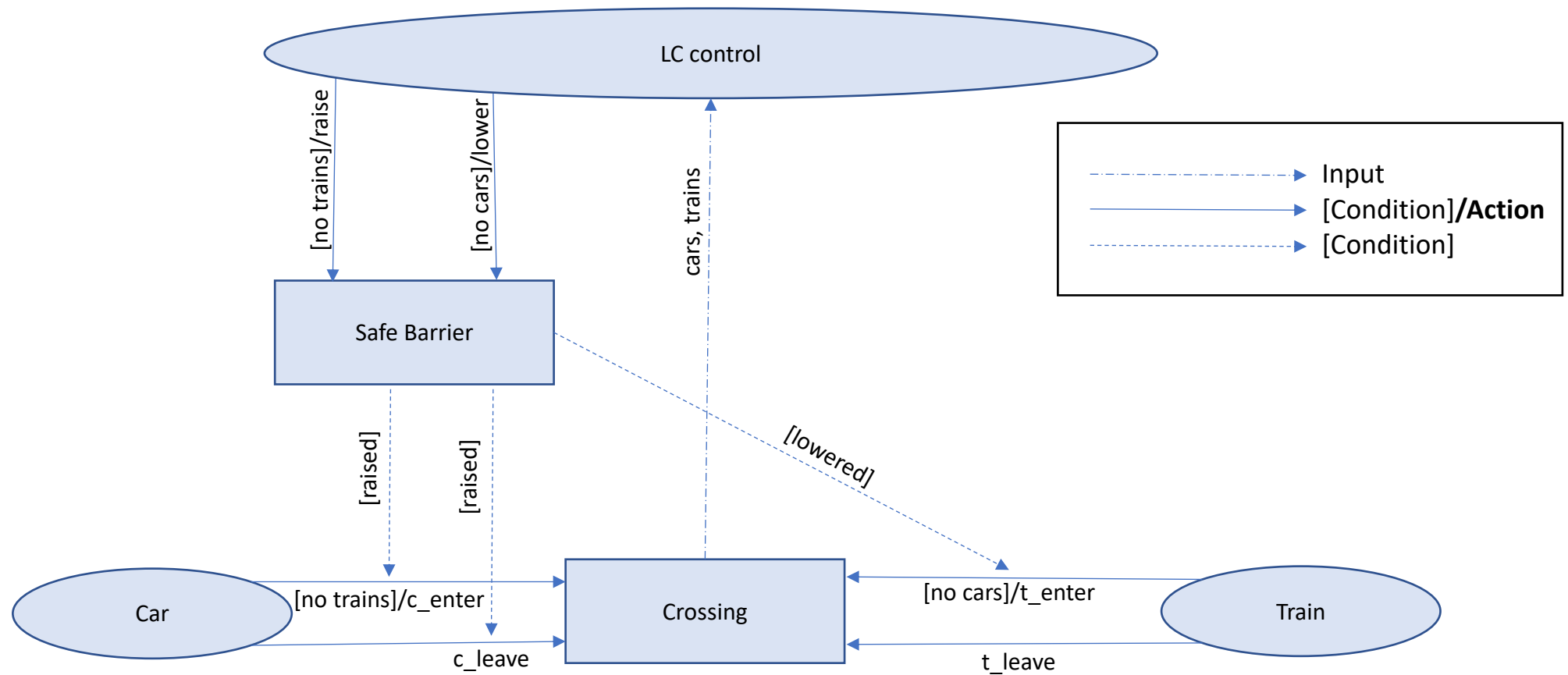
CarLeaveLC $\triangleq$

when *car* = on_crossing
then *car* := off_crossing

CarLeaveLC $\triangleq$

when *train* = on_crossing
then *train* := off_crossing

Level crossing: adding barrier



Refining Car Enter

Abstract *CarEnterLC* $\triangleq$

when *car* = *off_crossing* $\wedge$ *train* = *off_crossing*
then *car* := *on_crossing*

Refined *CarEnterLC* $\triangleq$

when *car* = *off_crossing* $\wedge$ *barrier* = *raised*
then *car* := *on_crossing*

- Requires invariant:
 - *barrier* = *raised* $\Rightarrow$ *train* = *off_crossing*

Refining Train Enter

Abstract *TrainEnterLC* $\triangleq$

when *train* = *off_crossing* $\wedge$ *car* = *off_crossing*
then *train* := *on_crossing*

Refined *TrainEnterLC* $\triangleq$

when *train* = *off_crossing* $\wedge$ *barrier* = *down*
then *train* := *on_crossing*

- *Requires*
 - *barrier* = *down* $\Rightarrow$ *car* = *off_crossing*

Barrier Events

New *RaiseBarrier* $\triangleq$

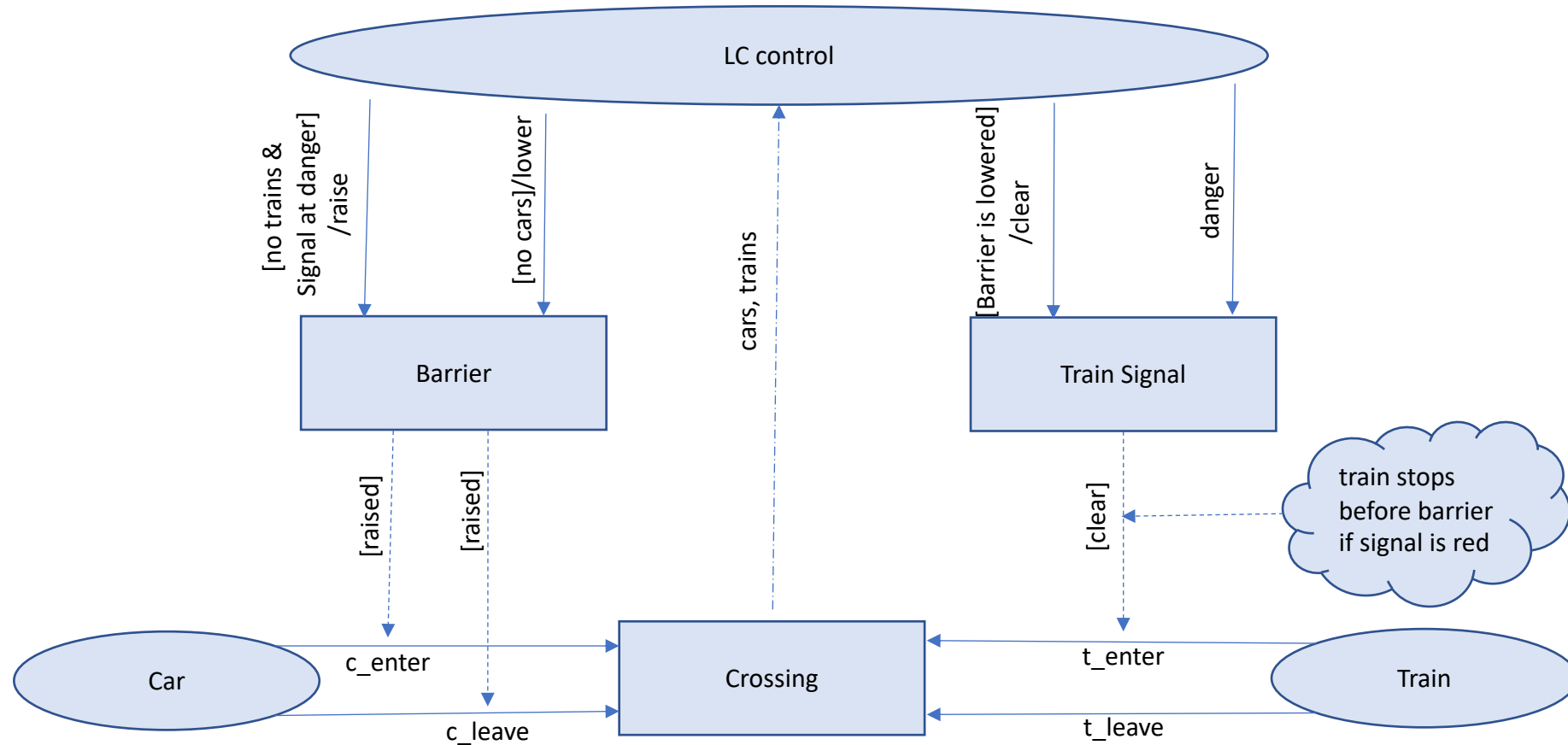
when $barrier = down \wedge train = off_crossing$
then $barrier := raised$

New *DropBarrier* $\triangleq$

when $barrier = raised \wedge car = off_crossing$
then $barrier := down$

- Guard discovery guided by need to maintain invariants:
 - $barrier = raised \Rightarrow train = off_crossing$
 - $barrier = down \Rightarrow car = off_crossing$

Level crossing: adding train signal



Refining Train Enter

Previous $\text{TrainEnterLC} \triangleq$

when $\text{train} = \text{off_crossing}$ $\wedge$ $\text{barrier} = \text{down}$
then $\text{train} := \text{on_crossing}$

Refined $\text{TrainEnterLC} \triangleq$

when $\text{train} = \text{off_crossing} \wedge \text{train_in_zone} \wedge \text{not}(\text{train_stopping})$
then $\text{train} := \text{on_crossing}$

- Requires invariant:

- $\text{train_in_zone} \wedge \text{not}(\text{train_stopping}) \Rightarrow \text{barrier} = \text{down}$

Train Passes Signal

New *TrainPassRed* $\triangleq$

when $\text{not}(\text{train_in_zone}) \wedge \text{signal} = \text{red}$
then $\text{train_in_zone} := \text{true} \mid \mid \text{train_stopping} := \text{true}$

New *TrainPassGreen* $\triangleq$

when $\text{not}(\text{train_in_zone}) \wedge \text{signal} = \text{green}$
then $\text{train_in_zone} := \text{true} \mid \mid \text{train_stopping} := \text{false}$

- Need to maintain invariant:
 - $\text{train_in_zone} \wedge \text{not}(\text{train_stopping}) \Rightarrow \text{barrier} = \text{down}$
- Requires this invariant
 - $\text{signal} = \text{green} \Rightarrow \text{barrier} = \text{down}$

Make signal green

New *SignalToGreen* $\triangleq$

when $signal = red \wedge barrier = down$
then $signal := green$

New *SignalToRed* $\triangleq$

when $signal = green$
then $signal := red$

- Need to maintain invariant:
 - $signal = green \Rightarrow barrier = down$

Move Barrier

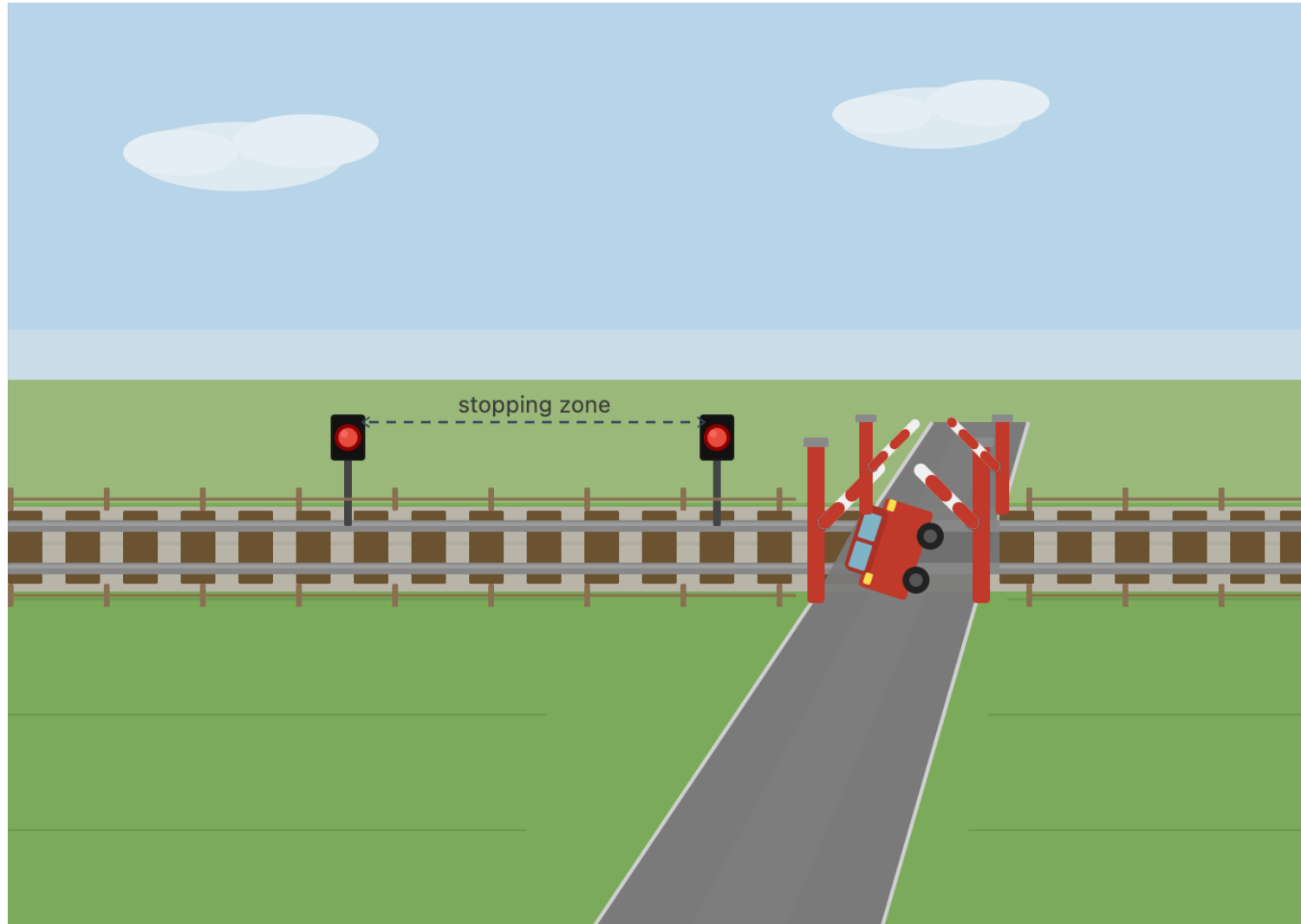
New *RaiseBarrier* $\triangleq$

when $barrier = down \wedge train = off_crossing$
then $barrier := raised$

Refined *RaiseBarrier* $\triangleq$

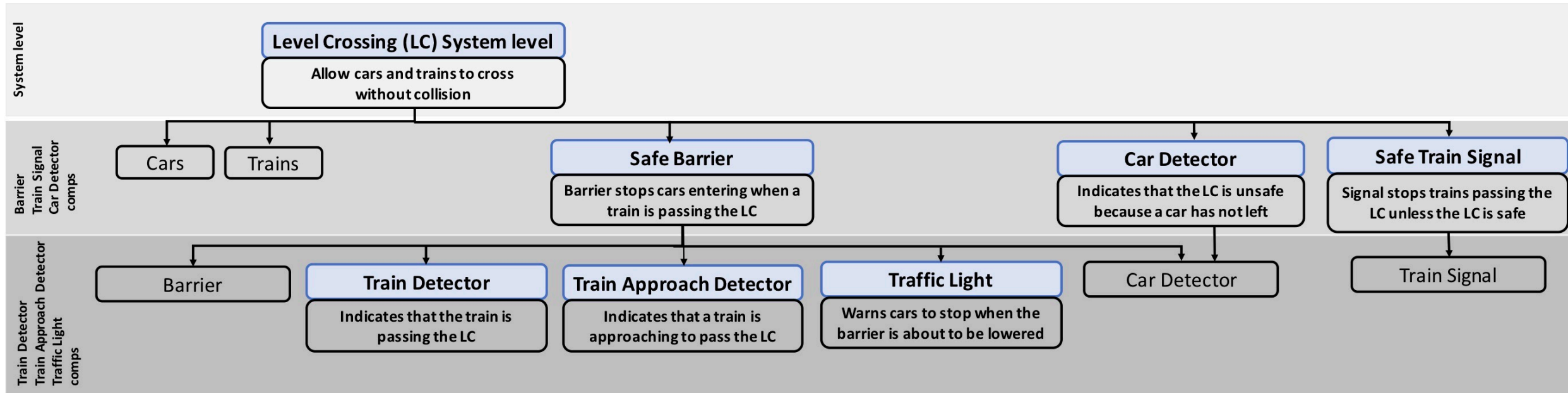
when $barrier = down \wedge train = off_crossing \wedge$
 $not(train\ in\ zone) \wedge signal = red$
then $barrier := raised$

- Need to maintain invariant:
 - $train\ in\ zone \wedge not(train\ stopping) \Rightarrow barrier = down$
 - $signal = green \Rightarrow barrier = down$



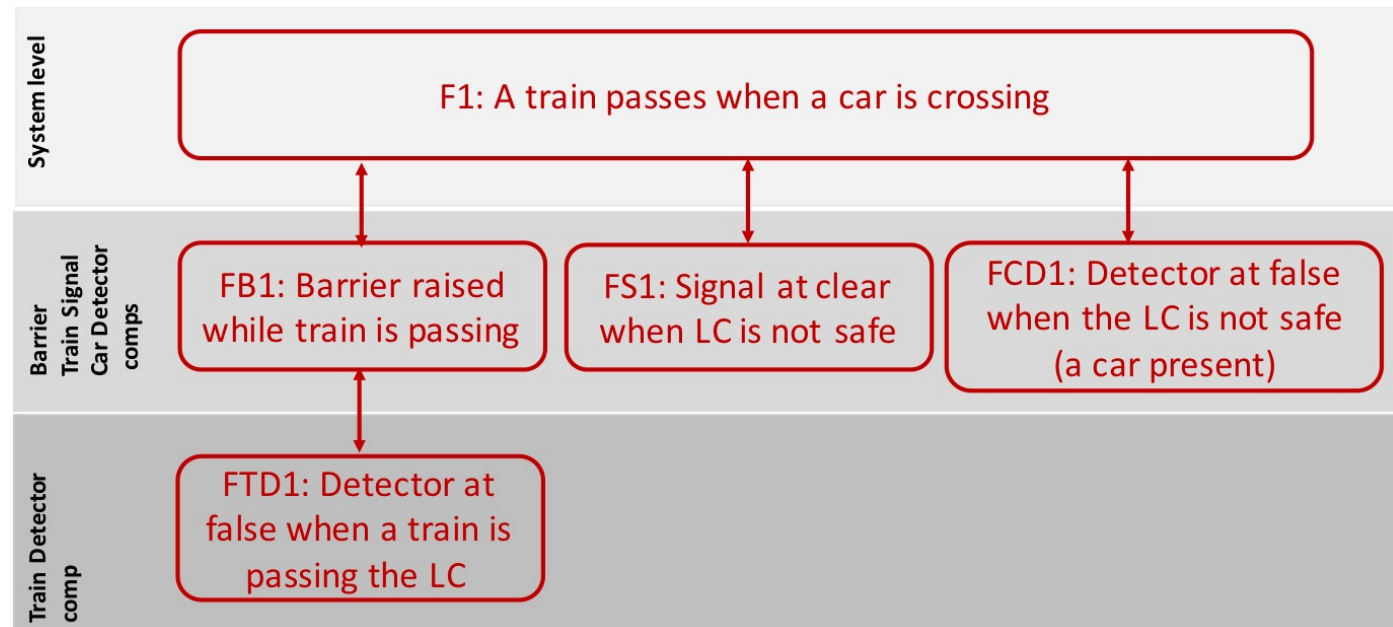
Other failures are possible

Hierarchy of LC safety requirements



Invariant analysis and fault trees

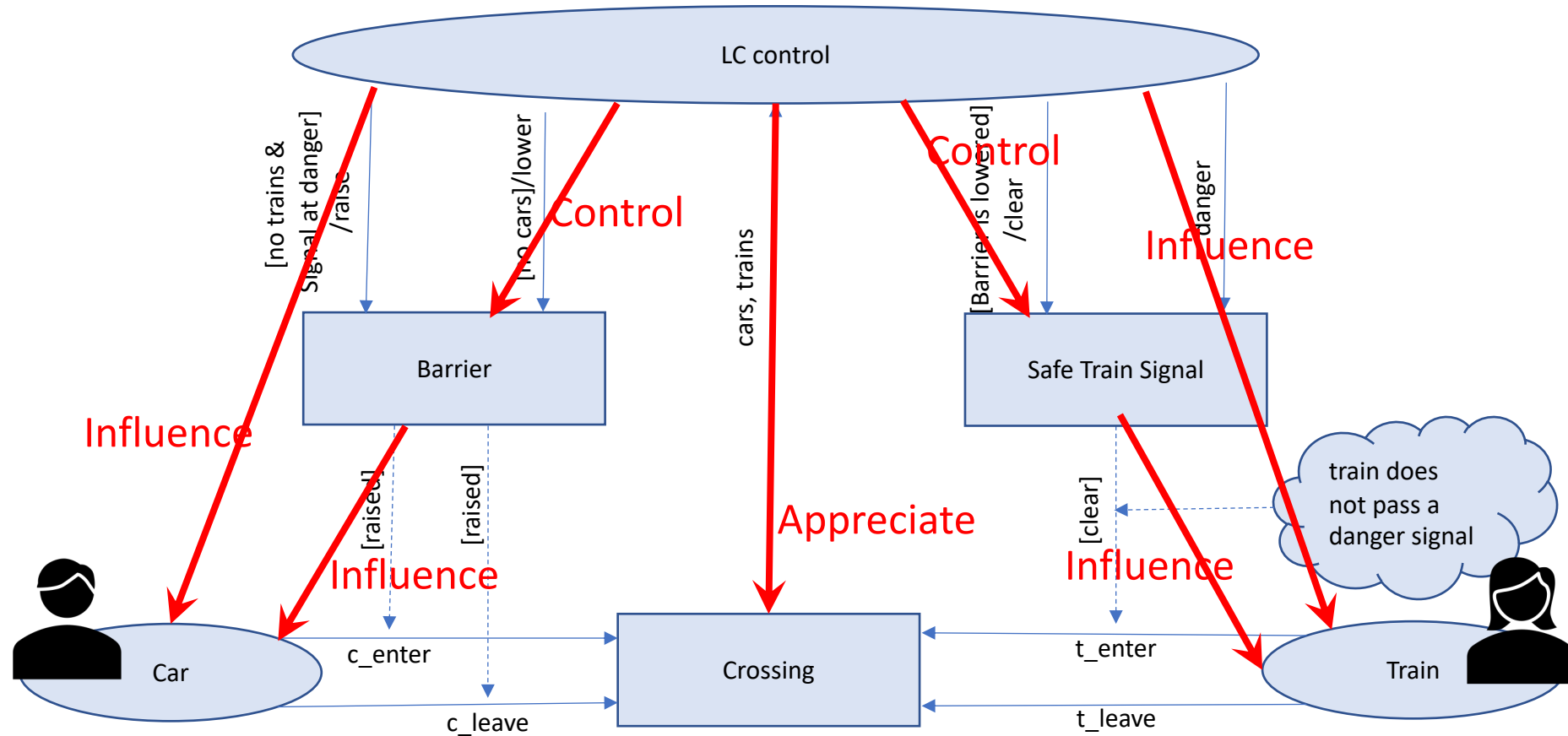
High-level Events	Occurring causes failure
Train enters LC	(F1) Train enters LC when a car is on crossing
Train leaves LC	No failure
Car enters LC	(F2) Car enters LC when a train is passing
Car leaves LC	No failure



Control and Autonomy

- Conventional controller model:
 - **Monitored** phenomenon (variables)
 - **Controlled** phenomenon (variables)
 - Yeganefard, Butler, Rezazadeh, *Evaluation of a Guideline by Formal Modelling of Cruise Control System in Event-B*, NFM 2010
- **AIC** relationships in social systems
 - **Appreciation**: entities that affect an agent in achieving their purpose
 - **Influence**: entities an agent can influence to achieve purpose
 - **Control**: entities an agent has power to control to achieve purpose
 - Smith, W.E. *Wholeness: Dimensions of Power*. Systemic Practice and Action Research (1999)
 - Al-Shareefy, Butler, Hoang, *AIC Approach for Intelligent Systems Requirements Elicitation*, Intl Conf on System Reliability and Safety (ICSRS), 2023

AIC in Level crossing



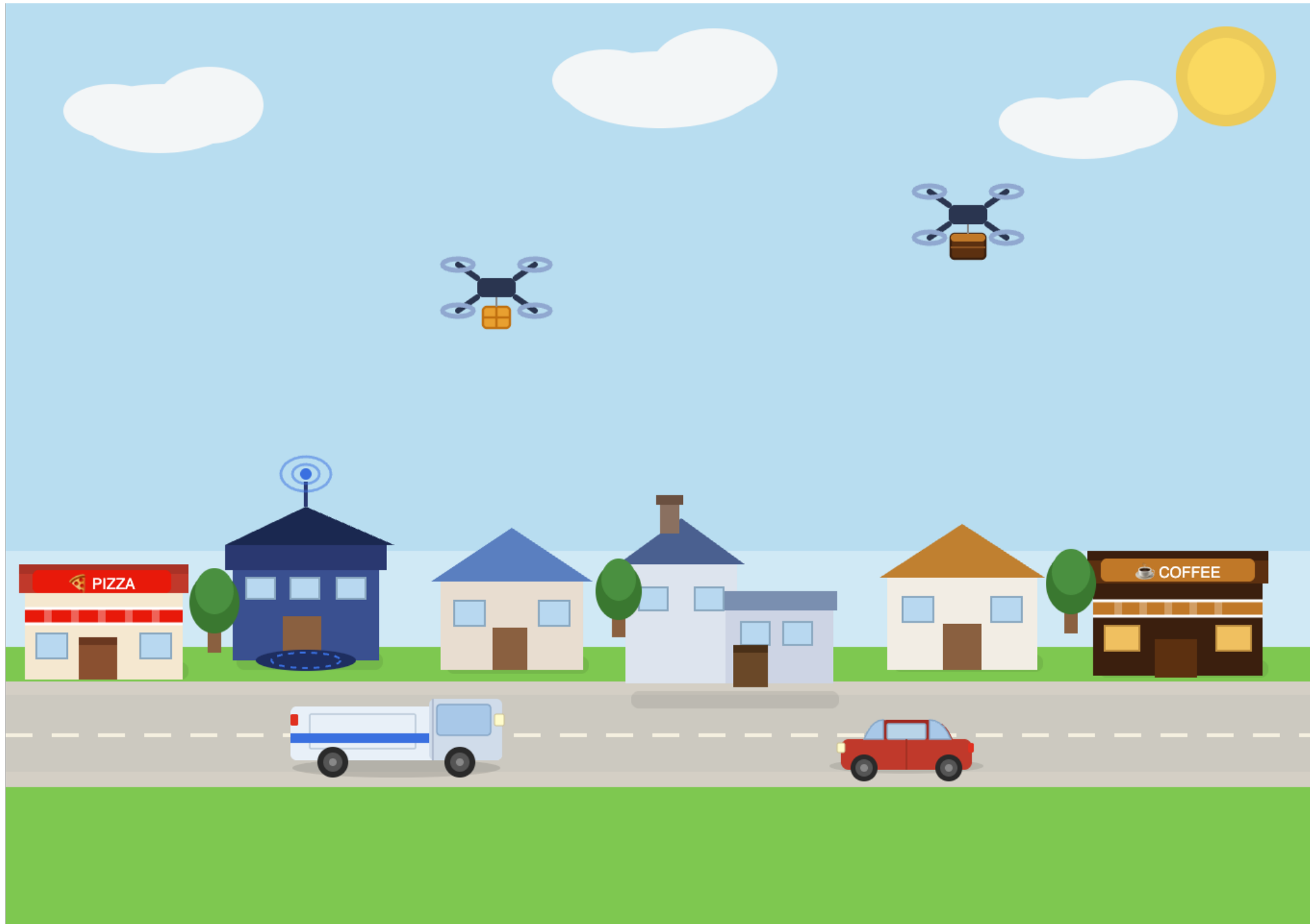
Examples of weaker influence mechanisms

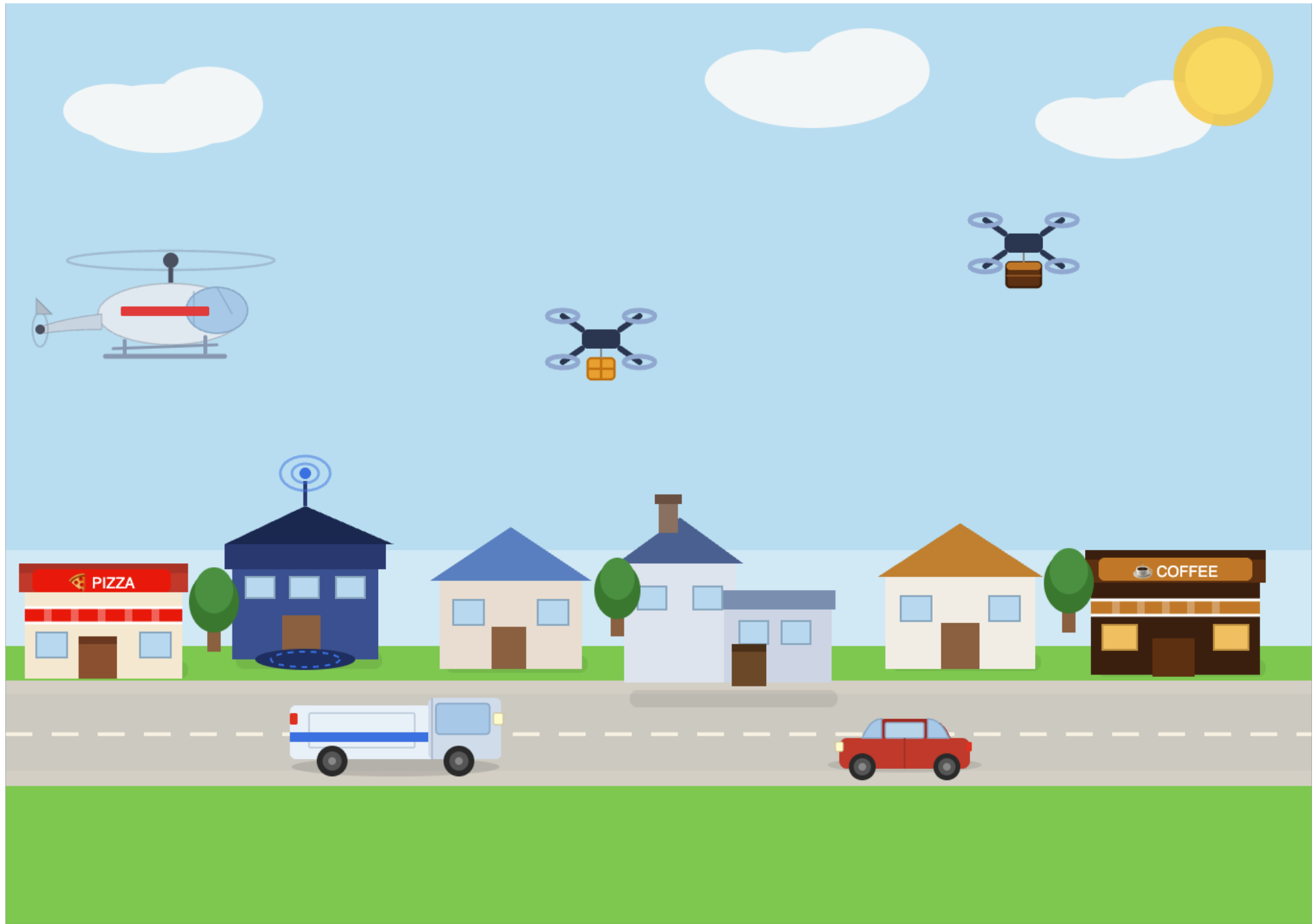


© Marc Pieters

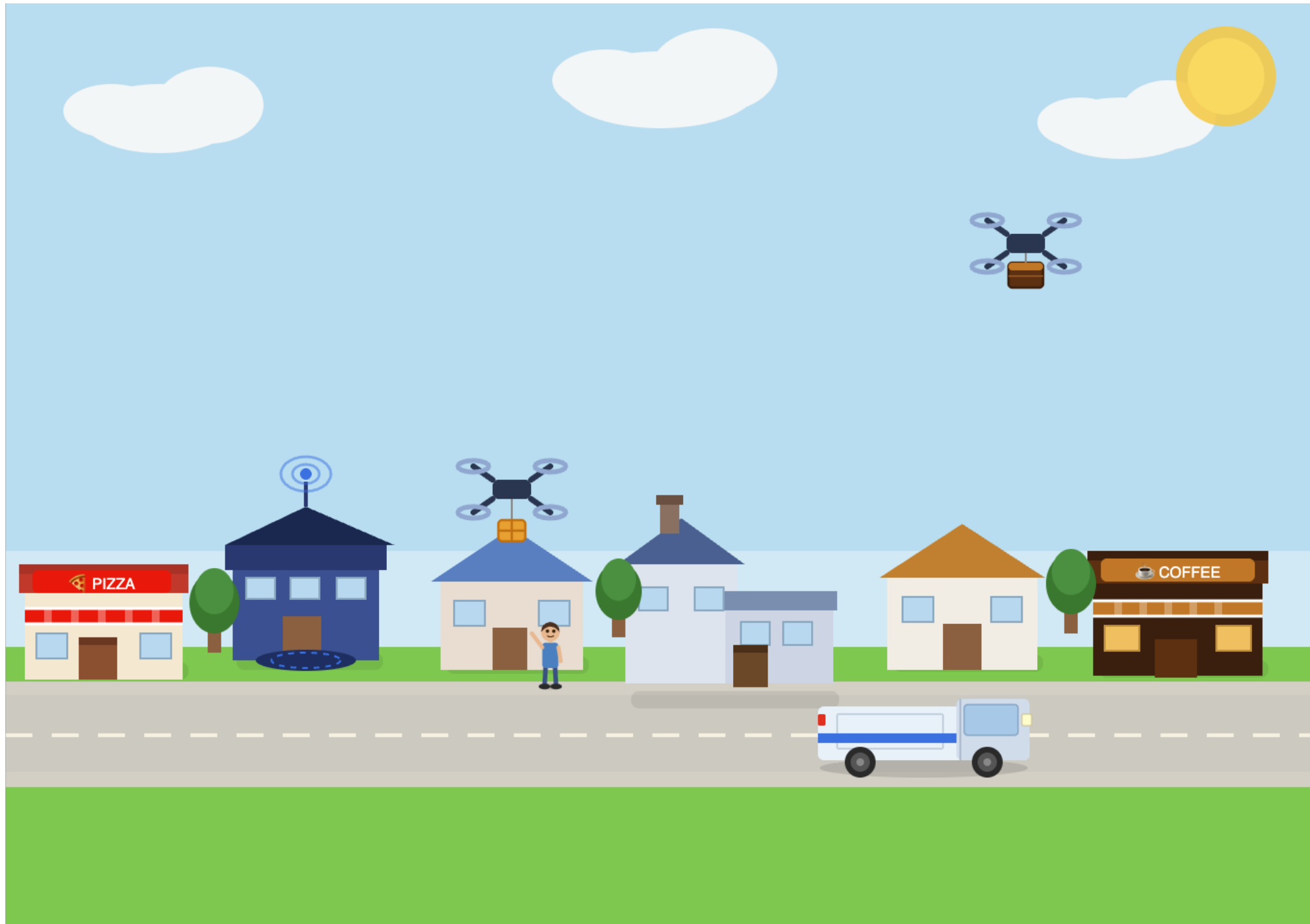


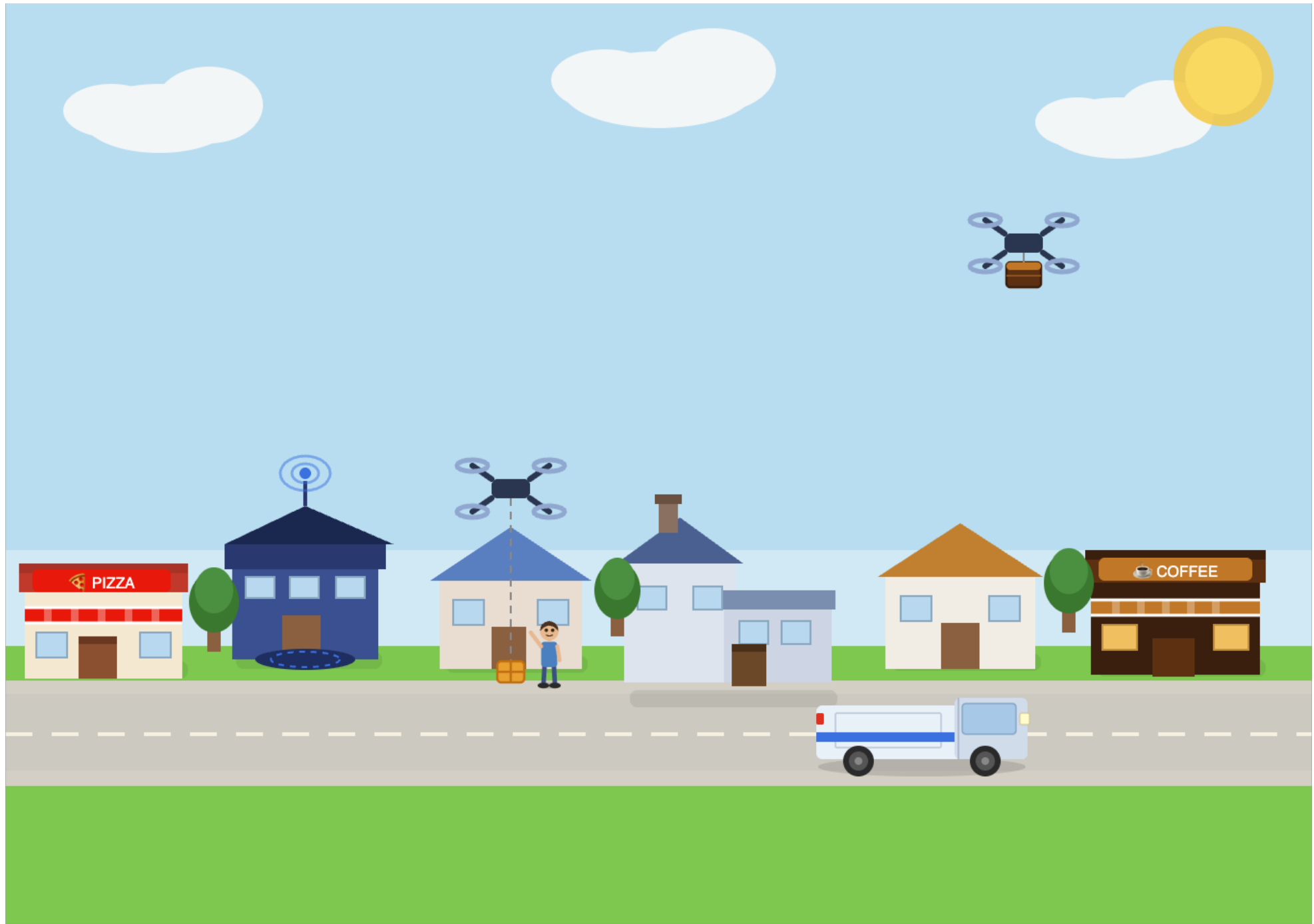
Autonomous Drone Delivery Service

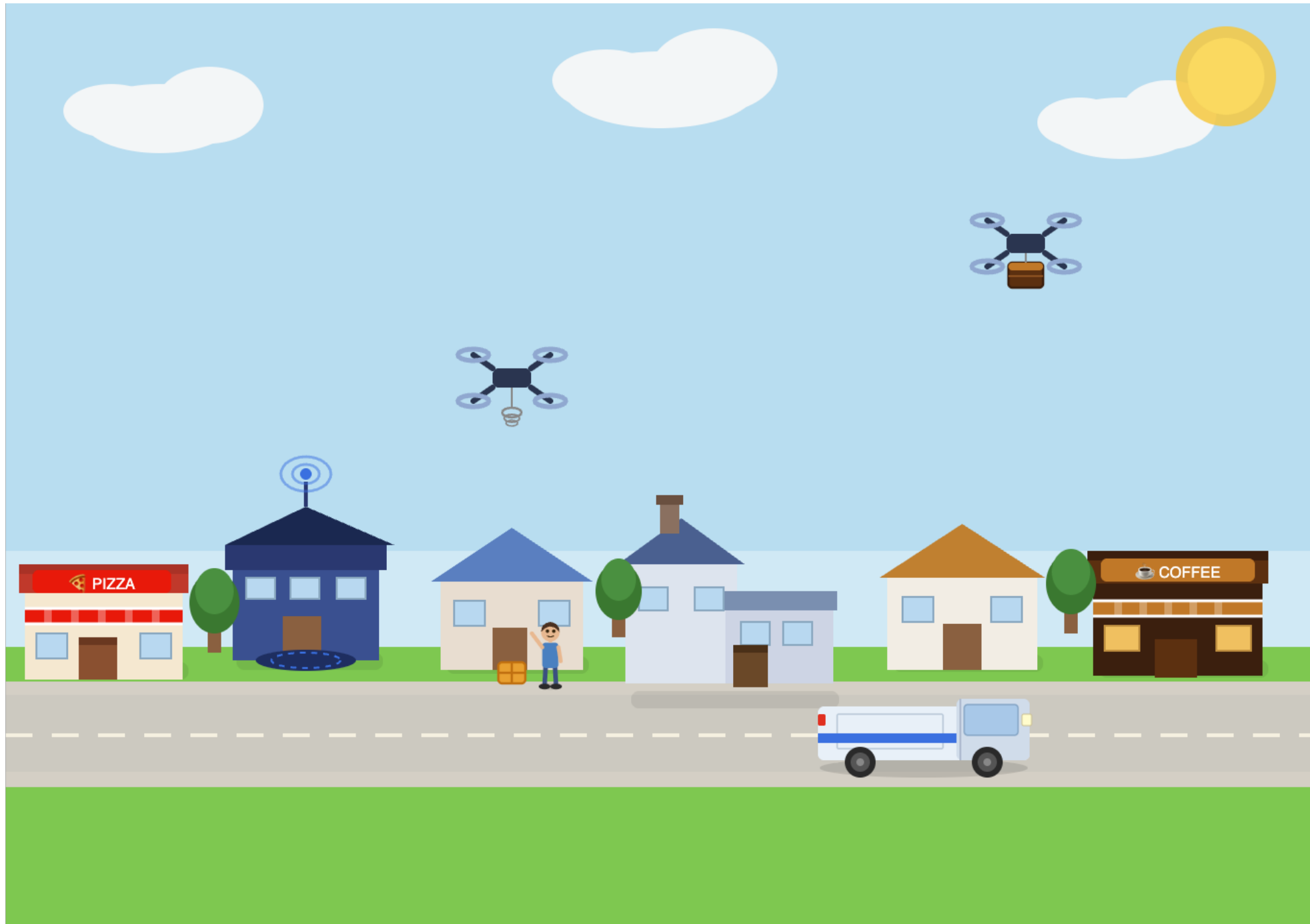


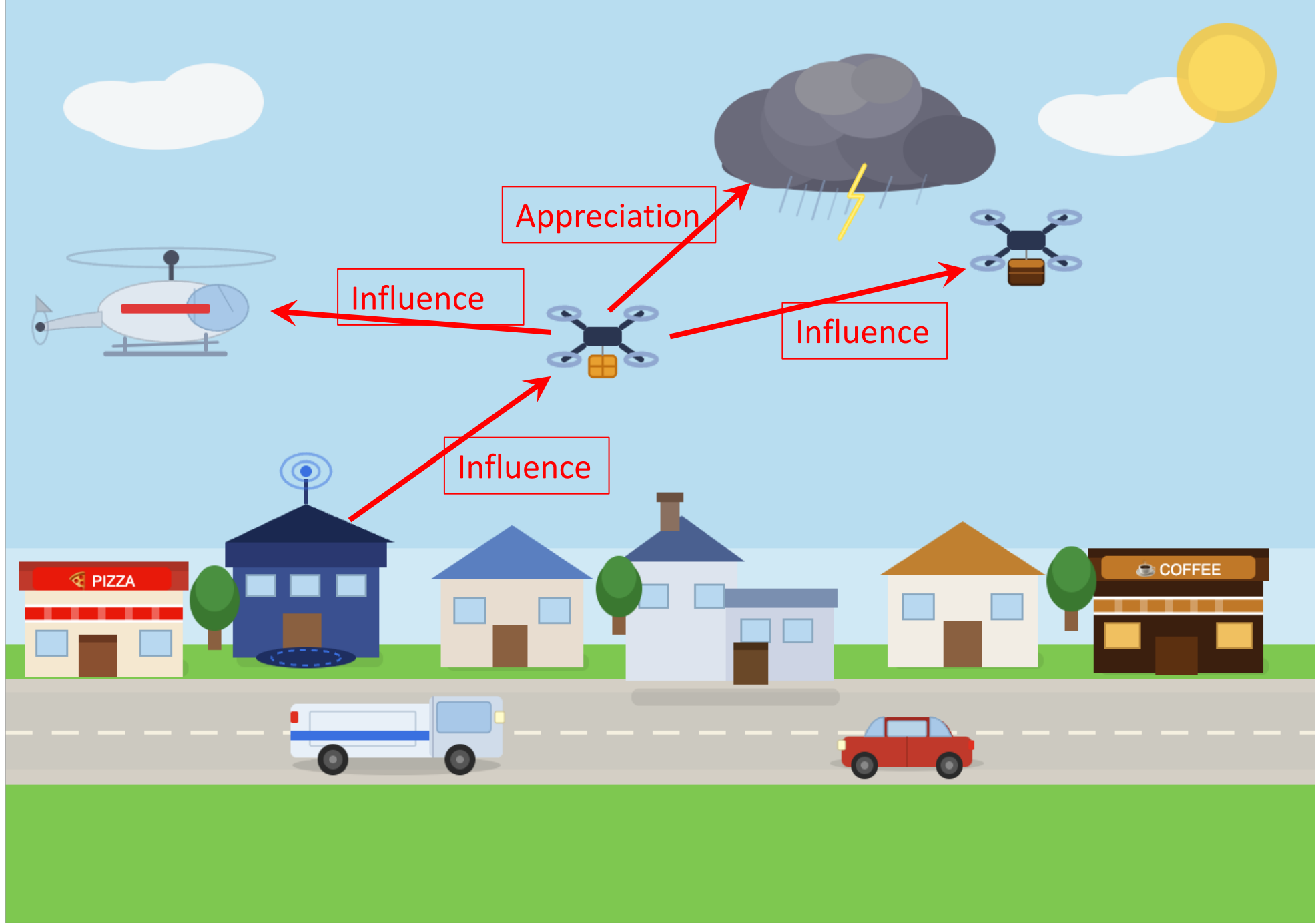


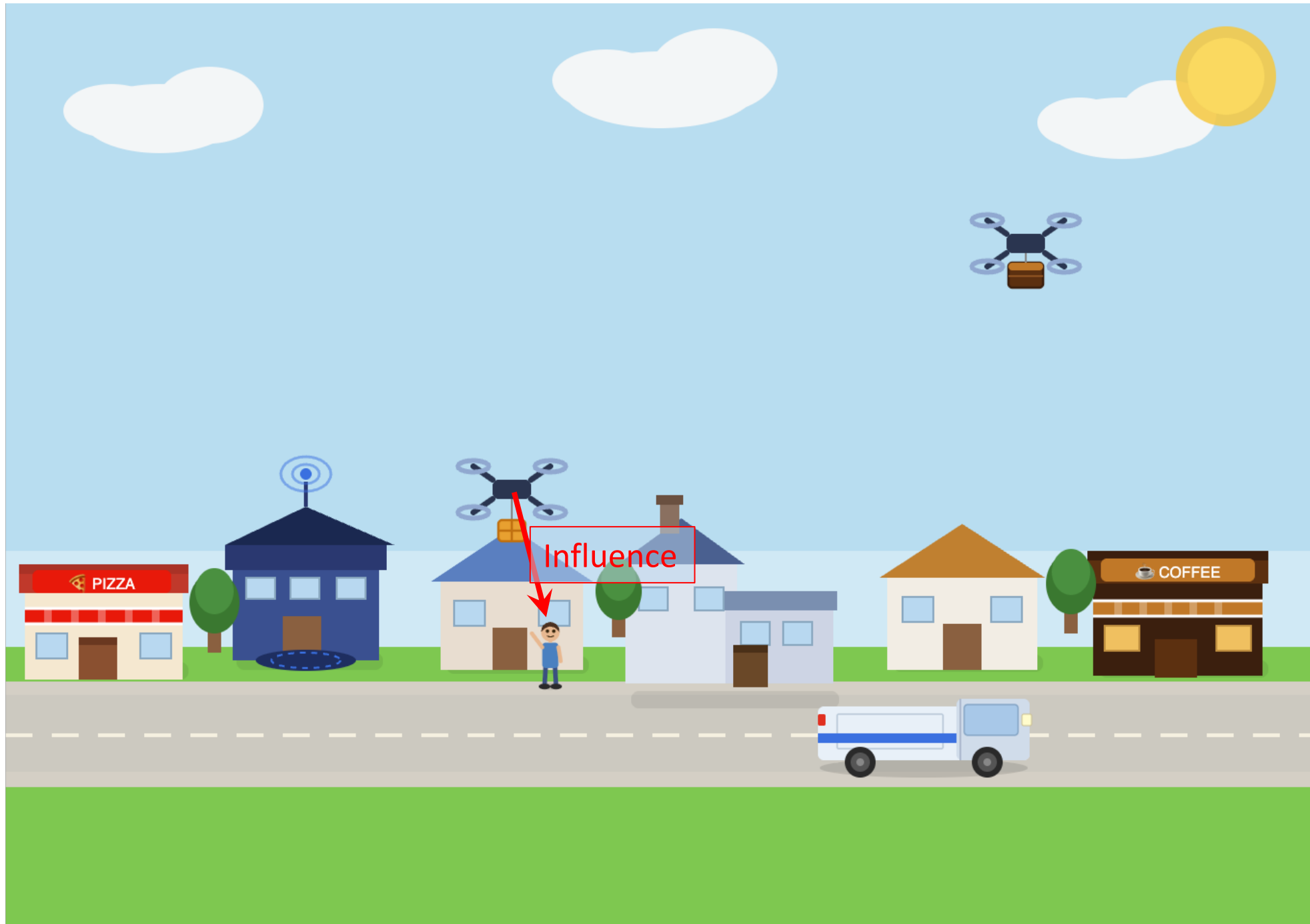




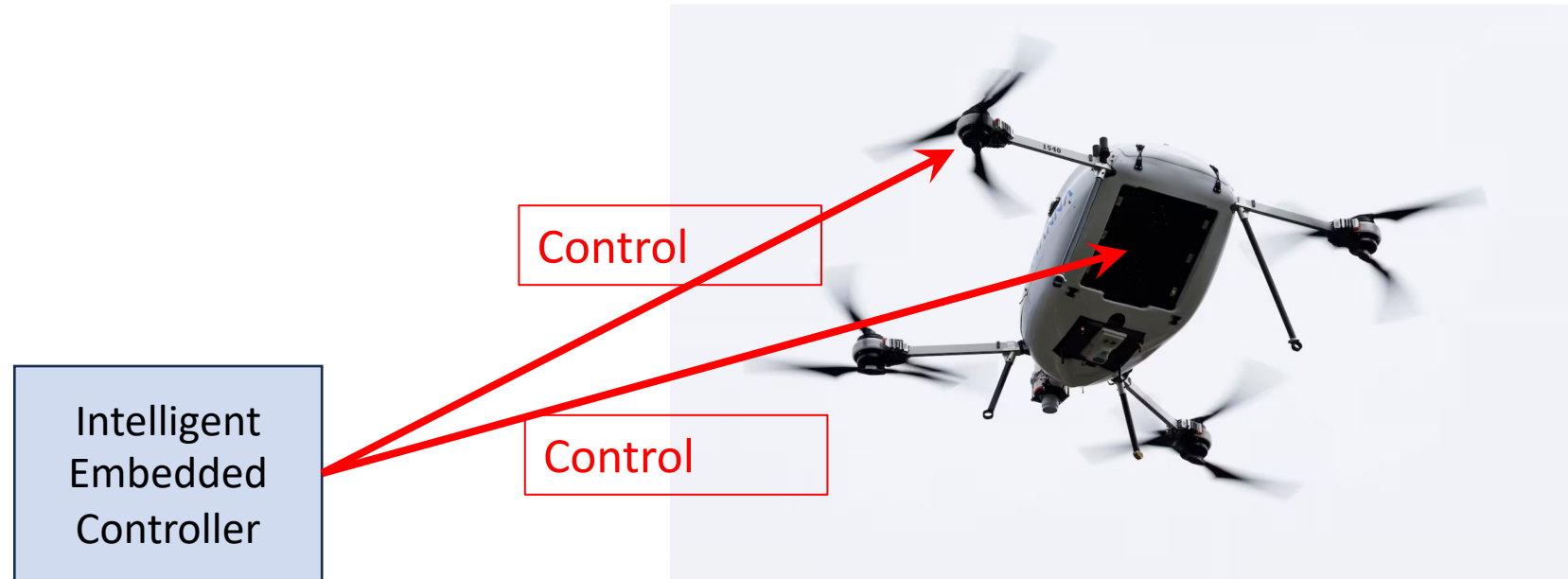








Embedded control



AIC Implications in Design

- **A controls B:** tight coupling between component A sensing and actuating properties of B
- **A influences B:** component A can signal to B but relies on assumptions about how/whether B will respond and sensing to check response
- **A appreciates B:** component A uses
 - static knowledge of B (e.g., B is a building) or
 - sensing to capture properties of B (e.g., B is a storm cloud)

Drone purpose: Complete a mission safely

- Mission:
 - Fly from station to retailer
 - Collect package from retailer
 - Fly to buyer property
 - Drop package at buyer property
 - Return to a station
- Safely:
 - Drone should not collide with other objects (structures, craft, humans) nor with ground
 - Package should only be dropped on clear flat area

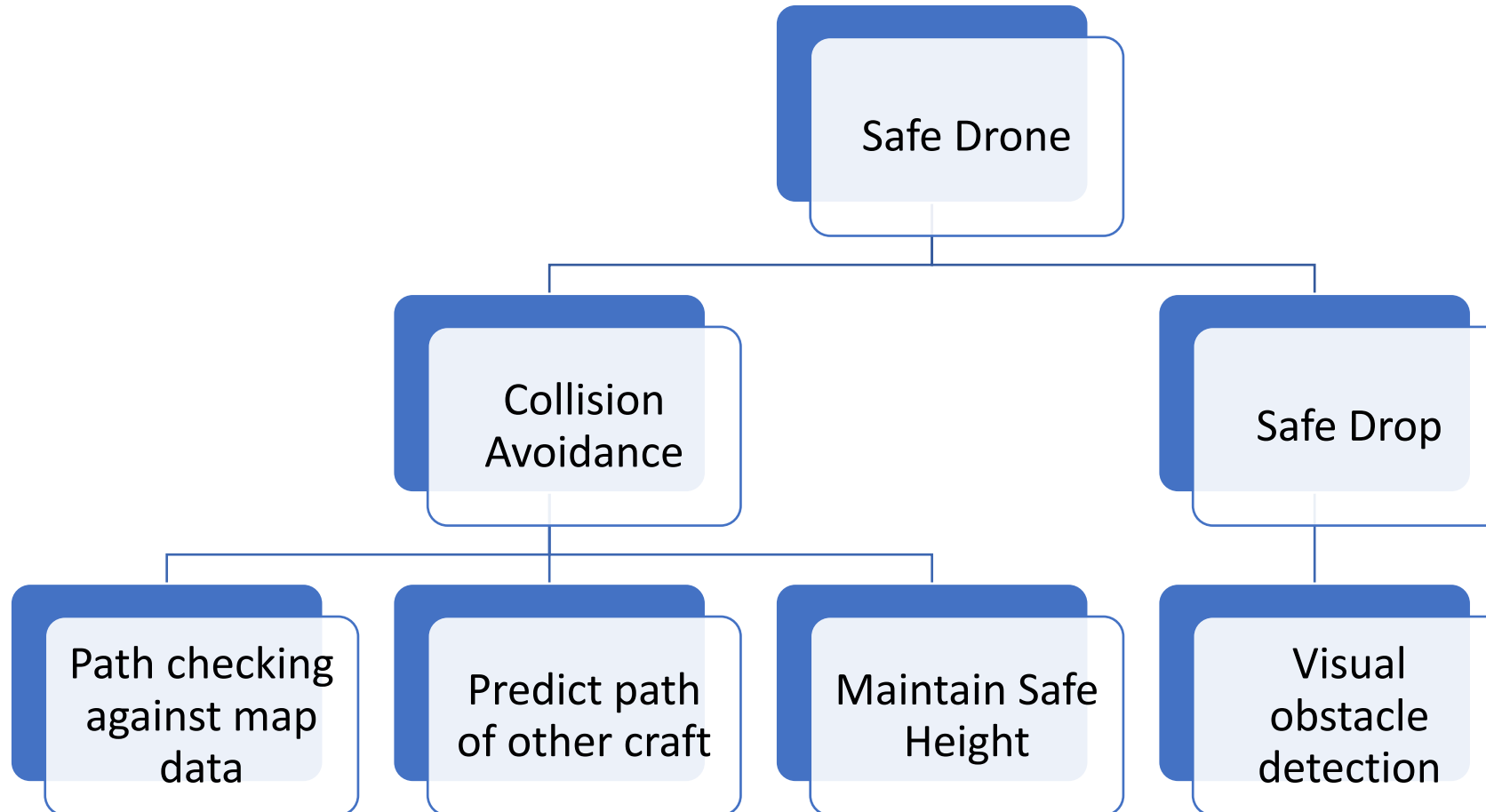
Failures

- Drone should not collide with other objects while flying
 - F1: drone collides with building
 - F2: drone collides with craft
 - F3: drone collides with human
- Drone should not collide with ground
 - F4: drone collides with ground
- Package should only be dropped on clear flat area
 - F5: package is dropped when area is not clear of obstacles/humans

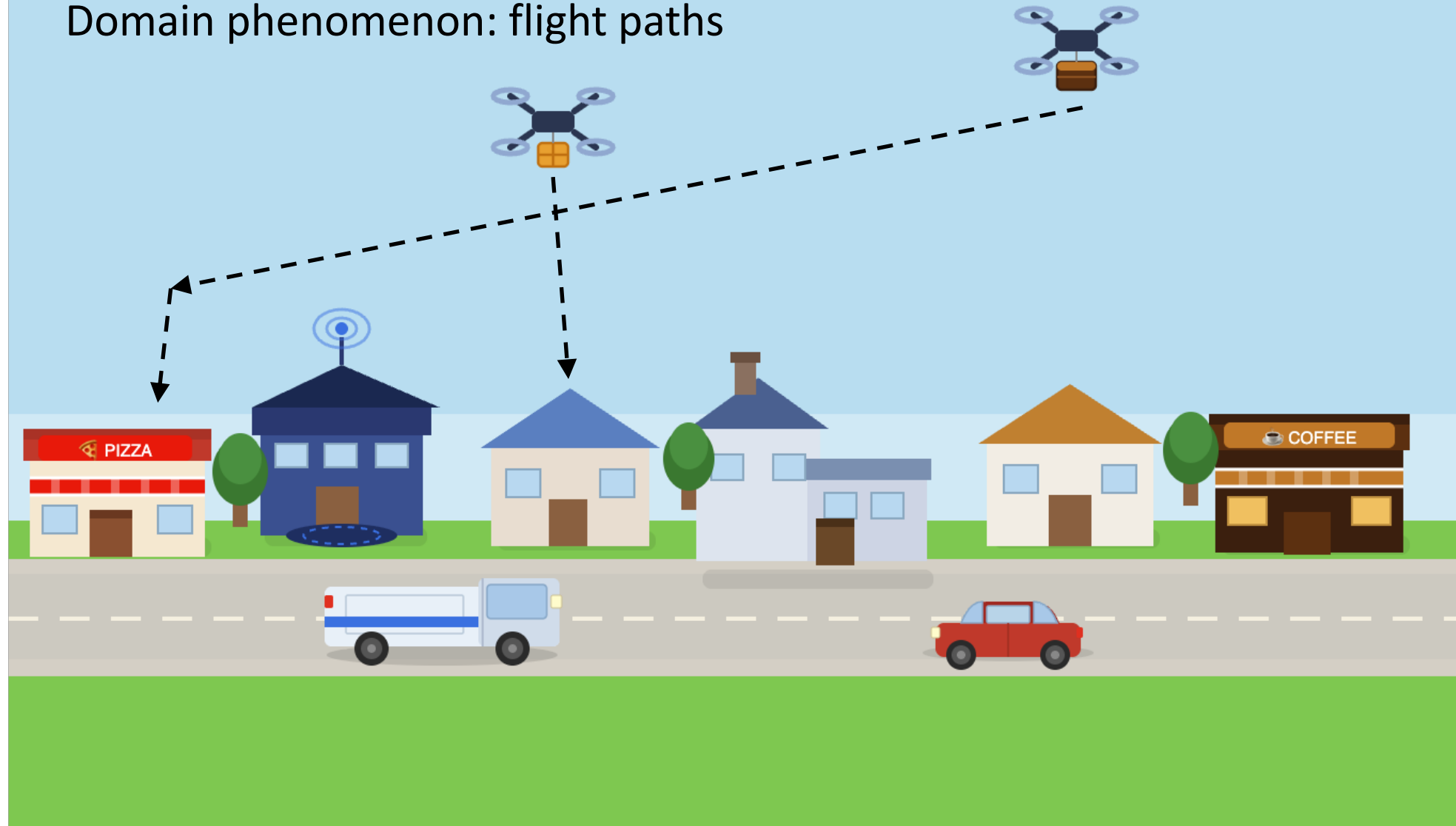
Failure mitigations

- F1: drone collides with building
 - M1: Advance collision avoidance checking of drone path against mapping data
- F2: drone collides with craft
 - M2: Awareness of flight path of nearby craft
- F3: drone collides with human
 - M3: Maintain minimum height about ground and buildings
- F4a: drone collides with ground due to loss of stability
 - M4a: Maintain flight stability
- F4b: drone collides with ground due to loss of power
 - M4b: Deploy parachute
- F5: package is dropped when area is not clear of obstacles/humans
 - M5: Visual obstacle detection

Proposed refinement strategy



Domain phenomenon: flight paths



Conflict free paths for drones

- **Path**: continuous function of time to 3-D space
- Two paths are **conflict-free** when at all times, the distance between them is at least the **minimum separation** (ms)

$$\text{conflict_free}(p1, p2, ms) = \\ \forall t. t \in \text{TIME} \Rightarrow ms \leq \text{dist}(p1(t), p2(t))$$

- C Bogdiukiewicz, M Butler, TS Hoang, M Paxton, J Snook, X Waldron, T Wilkinson, *Formal development of policing functions for intelligent systems*, International Symposium on Software Reliability Engineering (ISSRE) 2017

Refinement of paths: Waypoints, routes, segments

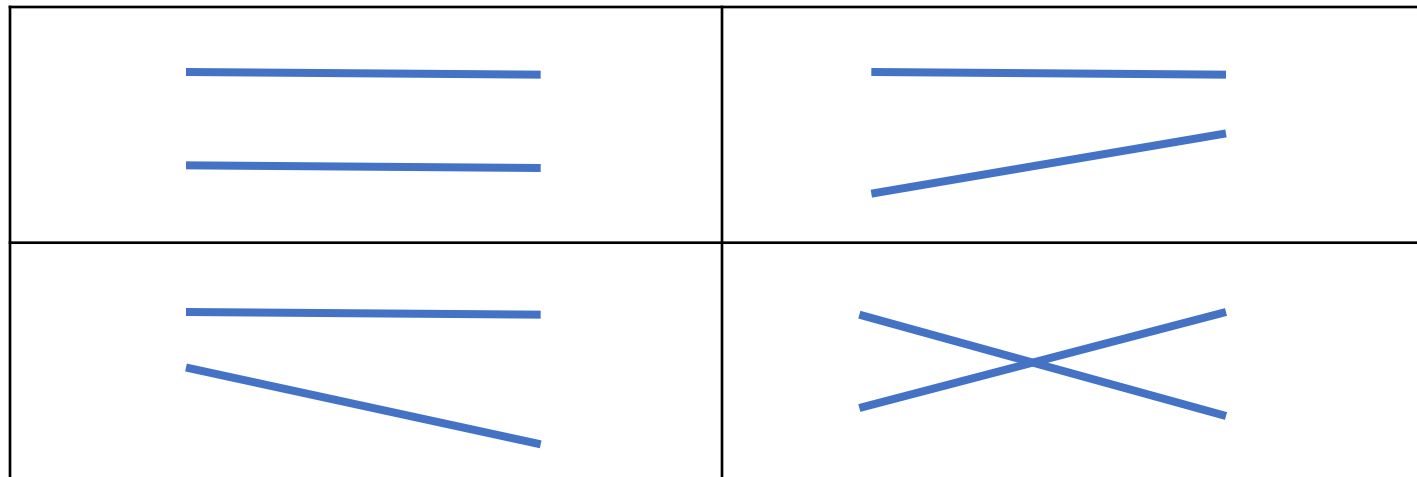
- **Waypoint**: time and 3-D position
- **Route**: sequence of waypoints increasing in time
- **Segment**: straight line path between successive waypoints on a route

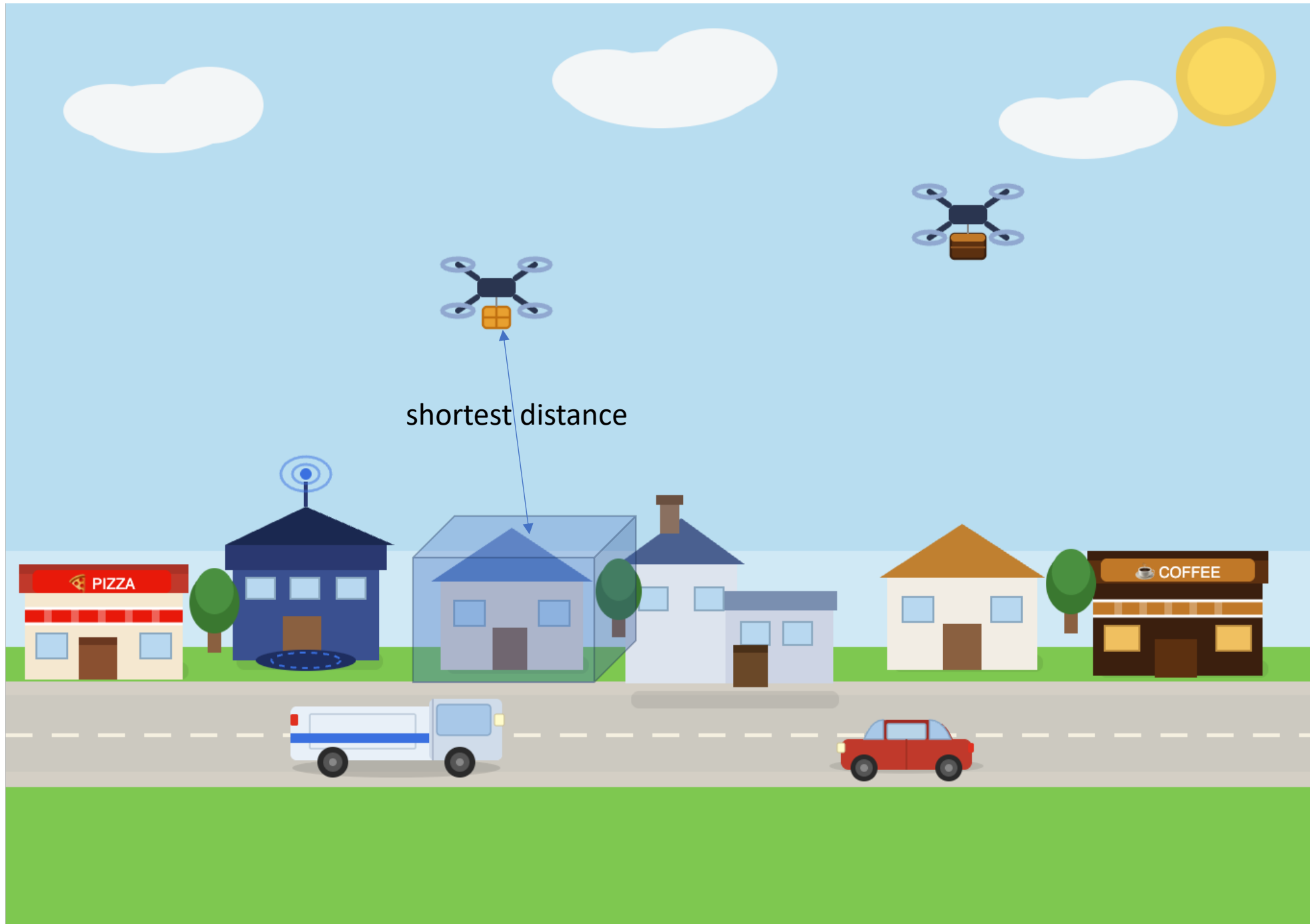
Checking linear segments

- **Abstract** spec involves universal quantification over real interval:

$$\forall t. t \in \text{TIME} \Rightarrow ms \leq \text{dist}(p1(t), p2(t))$$

- **Refined** to geometric argument on linear segments





Conflict free path and object

$\text{conflict_free}(p, o, ms) =$
 $\forall t \cdot t \in \text{TIME} \Rightarrow ms \leq \text{shortest_distance}(p(t), o)$

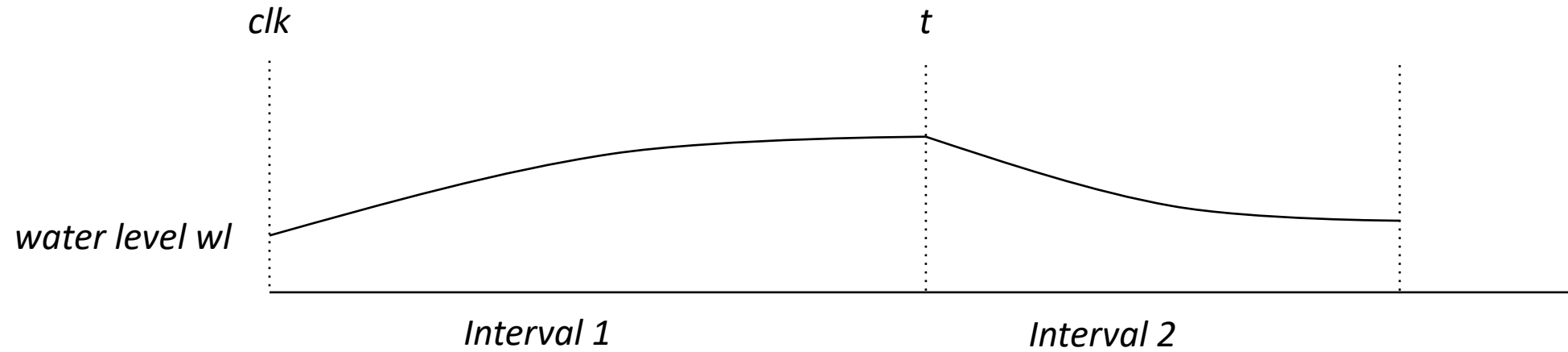
Path p

3D object o

ms minimum separation

Modelling Continuous Behaviour

- MJ Butler, JR Abrial, R Banach, *Modelling and Refining Hybrid Systems in Event-B and Rodin*, From Action Systems to Distributed Systems, 2016
- variable $\text{clk} \in \mathbb{R}^+$
- variable $f \in \mathbb{R}^+ \rightarrow \mathbb{R}$
- invariant $f \in \text{cts}(0, \text{clk})$ // f is a continuous function
- invariant $\text{ran}(f) \subseteq L..H$ // f is bounded
- Invariants are used to specify a property satisfied by the entire evolution of continuous variables.
- Interval events are used to specify a property on continuous evolution within an interval of nondeterministic duration.



invariant $\text{ran}(wl) \subseteq L..H$

event IncreaseWaterLevelInterval
any t, f where

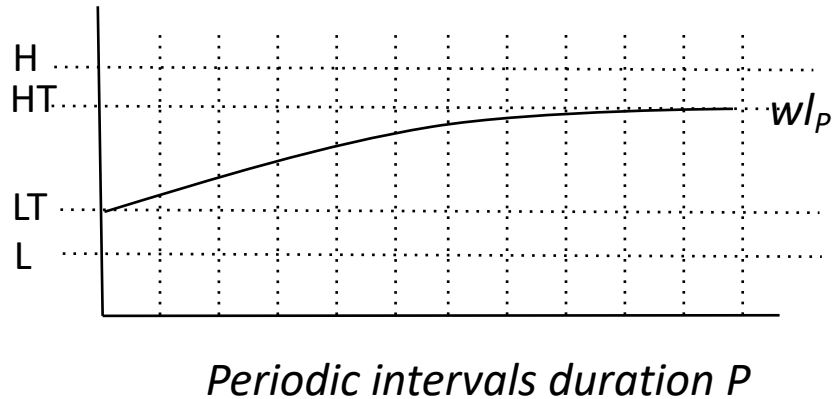
$t \geq \text{clk} + \epsilon$
 $f \in \text{cts}(\text{clk}, t)$
 $f(\text{clk}) = wl(\text{clk})$
 $f \in \text{mono_inc}$
 $f(t) \in L..H$

then

$wl := wl \cup f$
 $\text{clk} := t$

end

Refine by periodic control of pump



(refined) event IncreaseWaterLevelInterval

pump = on
 $wls(\text{clkm}) > HT$

then

pump := off

end

new event PeriodicIntervalIncrease

any f_p where

pump = on

$wl_p(\text{clk}_p) \leq HT$

$f_p \in \text{cts}(\text{clk}_p, \text{clk}_p + P)$

$f_p(\text{clk}_p) = wl_p(f_p)$

$f_p(\text{clk}_p + P) \leq wlm(\text{clk}_p) + (RI \times P)$

$f_p \in \text{mono inc}$

then

$wl_p := wl_p \cup f_p$

$\text{clk}_p := \text{clk}_p + P$

end

axm1 : $L < LT < HT < H$

axm2 : $L \leq LT - (RD \times T)$

axm3 : $HT + (RI \times T) \leq H$

RI: rate of increase when pump on

Refining segment interval events

- Periodic control of pump: introduce new events to break large nondeterministic continuous intervals to smaller period intervals
- Effect of this refinement is to resolve abstract nondeterminism in earlier stages
- This allows to model actions that prevent violation of constraints on abstract continuous intervals, e.g.,
 - Choose path that avoids known static obstacles
 - Change path if dynamic object detected and predicted to lead to conflict

Guaranteed Safe AI again

- Socio-technical benefit of GS AI: *“it makes democratic oversight easier, because concrete safety specifications can be audited and discussed by outside observers and regulators”*
- *“...if it is impossible to create world models that are sufficiently accurate to ensure that an AI system adheres to some specification, then it is presumably in general impossible to ensure that this specification is satisfied by that system.”*

Towards Guaranteed Safe AI: A Framework for Ensuring Robust and Reliable AI Systems
Dalrymple, Skalse, Bengio, Russell, Tegmark, Seshia, Omohundro, Szegedy, Goldhaber,
Ammann, Abate, Halpern, Barrett, Zhao, Tan, Wing, Tenenbaum

References

- Fathabadi, Snook, Dghaym, Hoang, Alotaibi, Butler. *Systematic hierarchical analysis of requirements for critical systems*, Innovations Syst Softw Eng 2024
- Salehi Fathabadi, Asieh, Hoang, Thai Son and Butler, Michael, *SHARCS: refinement-centric hazard analysis of requirements for critical systems*, ABZ 2026
- Yeganefard, Butler, Rezazadeh, *Evaluation of a Guideline by Formal Modelling of Cruise Control System in Event-B*, NFM 2010
- Al-Shareefy, Haider, Butler, Michael and Hoang, Thai Son (2024) *AIC approach for intelligent systems requirements elicitation*, In 2023 7th International Conference on System Reliability and Safety, ICSRS 2023
- C Bogdiukiewicz, M Butler, TS Hoang, M Paxton, J Snook, X Waldron, T Wilkinson, *Formal development of policing functions for intelligent systems*, International Symposium on Software Reliability Engineering (ISSRE) 2017
- MJ Butler, JR Abrial, R Banach, *Modelling and Refining Hybrid Systems in Event-B and Rodin*, From Action Systems to Distributed Systems, 2016

Thank you